Self-Evolving Recommender Systems: A Survey and an Evaluation Audit

Shanglin Yang
Independent Researcher
USA
kudoysl@gmail.com

Zhan Shi
Santa Clara University
Santa Clara, CA, USA
ashi2@scu.edu

Abstract

Large language model (LLM) agents now improve recommenders the way engineers do: they edit the model's configuration or code, train the result, and keep what the evaluation favours. By 22 October 2026 we found 34 such loops (one known only from a blog post), from open-source harnesses to systems at Google, Meta, Amazon, Alibaba, Kuaishou and others. We organise them by what the loop changes, who proposes the change, and which signals decide whether it survives. We then audit how 27 of them report their evaluation on eight criteria, and release the coding with its evidence. The reporting is thinnest where an optimiser most needs it. Of the 16 loops that select candidates offline, none reports the spread over repeated sessions, eight do not say which data select the candidates (seven of these confirm the kept candidate online), and two state that the test set was used only once. Of the 10 loops that select by online experiment, so that the reported effect is the best of several rounds, none fully corrects for testing many rounds and one confirms its chosen candidate in a fresh experiment (coded by one author). Four loops report shortcuts in their own evaluation, such as a metric inflated by a changed batch size in an exploratory session. The counts describe what the papers report, as coded by the authors, not independently. We turn them into a proposed two-tier reporting checklist whose minimum tier asks authors only to state what they did.

Keywords

recommender systems, LLM agents, AutoML, program evolution, evaluation, reproducibility

1 Introduction

Recommender models have long been improved by a loop that engineers run by hand: propose a change, train, compare offline, and launch the winners into an online experiment. Automating parts of this loop is an old goal. Neural architecture and embedding search, mostly for click-through-rate (CTR) prediction models [104, 159, 215, 220], and loss search [93] automated the choice inside a space that a person had written down [223]. What changed in the loops we found, nearly all from 2026 (our arXiv screen began in 2025), is that the proposer became an LLM agent that writes code. Following FunSearch [150], AlphaEvolve [131] and open implementations such as OpenEvolve [132] and ShinkaEvolve [83], agents now edit the model, loss and training code of recommenders directly. Google reported such a loop for YouTube ranking [170] and Self-EvolveRec released an open harness [78] in February 2026; within months, open harnesses such as RecHarness [102] and systems at Meta [90, 121], Alibaba [60, 123], Kuaishou [84, 192, 231], Tencent [4, 109], NetEase [213] and others followed.

These systems are optimisers, and an optimiser exploits whatever its objective rewards. A loop that evaluates hundreds of agent-written candidates on one validation set and keeps the maximum is the setting in which the known weaknesses of recommender evaluation—sampled metrics [82], temporal leakage [67], untuned baselines [32, 147], and the reuse of a holdout for adaptive decisions [13, 38]—are, we expect, the ones most readily turned into reported gains. The agent also controls the code that produces its score. Four of the audited loops and one comparison point report incidents in which a candidate raised the metric, or would have, without a real improvement; in all but EvoPilot, whose paper does not say who chose the leaky evaluation date, the agent produced it. For example, RecEvolve's agent, in an exploratory session, inflated its proxy metrics by shrinking the batch size, and with it the number of in-batch negatives [135], and an AI co-scientist for search ranking produced implementations that "improved $\mathcal{M}$ while being wrong (e.g., silently increasing model size)" [182]. Four incidents cannot estimate how often this happens, and our audit does not measure overfitting or gaming rates: it measures what papers report about the safeguards against them. Figure 1 shows the loop these systems share and the points where the evidence for a gain can fail.

No existing survey covers this development (§3 compares the nearest ones), and no audit has asked how its gains are evaluated. This paper does both, as a dated snapshot with a cut-off of 22 October 2026. Our contributions, in order of weight, are:

(1) **An evaluation audit** (§6). We code the 27 recommendation loops that keep or reject trained candidates on eight criteria, report loops that select offline and loops that select online separately, give a range for every count that rests on unstated practice, and catalogue the shortcuts agents found. Every code and its evidence are released.
(2) **A proposed reporting checklist** (§7). Its minimum tier asks authors only to state what they did; its recommended tier costs compute. Each item maps to an audit criterion, and we read the audit coding against it as a retrospective proxy.
(3) **A taxonomy and synthesis** (§4–§5) that separates what the loop changes, who proposes the change and which chain of signals decides whether it survives, and records the loops' recurring designs and what they actually kept.

Findings at a glance. The counts are over the 27 coded loops, in two strata (§6); each is explained, with its range and reliability, in §6.

- F1: 3 of the 10 offline-selecting loops that report ranking metrics rank the full catalogue (3–7 if read either way).
- F2: 8 of the 16 offline-selecting loops do not state their selection data, so 4–12 may select on the data they report.

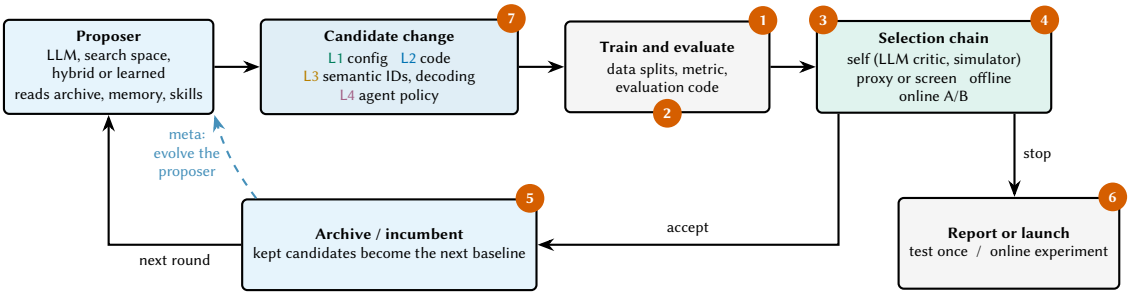


Figure 1: Anatomy of a self-evolving recommender. A proposer changes some part of the recommender (the evolved object); the changed system is trained and evaluated; a chain of selection signals decides what is kept and becomes the next baseline, and the loop may also change its own proposer (meta). Red markers show where, by design, the evidence for a reported gain can fail (possibilities, not measured failure rates), with the audit finding and criterion that check it: 1 the agent edits evaluation code, reads test data or games the metric (F7, C9); 2 sampled or unstated candidate sets (F1, C1); 3 selection on unstated or reported data, without a single test evaluation (F2, F3; C2, C3); 4 a proxy or self-generated signal that does not transfer (F5, C5); 5 a gain inside run-to-run noise becomes the new baseline (F4, C4); 6 online rounds selected without correction or fresh confirmation (F9, R9); 7 gains measured against an untuned starting point, with no equal-budget null search (F6, F8; C6, C8).

- F3: 2 of 16 state test-once use (2–10).
- F4: 0 of 16 report spread over repeated sessions.
- F5, F6: 1 of 16 checks its selected candidates on held-out data of the same task, and 3 of 16 compare against an equal-budget non-LLM search.
- F7: 4 of 27 report shortcuts in their own evaluation.
- F8: the 17 reported lifts on a broad online metric range from +0.052% to +4.86%.
- F9: of the 10 online-selecting loops, 0 correct across rounds (1 in part) and 1 confirms afresh (4 in part).

2 Scope and Survey Method

Definition. We call a recommender system *self-evolving* when an automated proposer changes some part of it, the changed system is evaluated, and the evaluation decides which changes are kept, repeatedly and without a human choosing each change. As a codable rule: a loop is in scope when an evaluation result decides which candidates survive and a person can at most gate, approve, veto or roll back what it kept, or review it at checkpoints; it is out of scope when a person chooses which candidates to keep without an evaluation deciding first. This includes systems whose proposer is a search algorithm over a fixed space (background, §3), but our focus is on LLM proposers.

Inclusion. We include (i) systems that change a recommender's or ranker's configuration, features, code or components under an evaluation loop; (ii) agentic or LLM recommenders whose policy, memory, skills, prompts or weights improve from feedback they generate; and (iii) benchmarks, environments and surveys for these. Only (i) is audited. The boundary is the selection step: in (i) a trained candidate recommender is evaluated and kept or rejected, which is what the audit criteria test, whereas in (ii) the recommender's own state is updated continuously from feedback it mostly generates itself, with no separate candidate to accept. We therefore catalogue (ii) as adjacent work (§D.5) but do not audit it. A loop qualifies for (i) whatever the size of the change, so loops that change only configuration values, such as GEARS's segment-level treatment policies chosen by online experiments, are audited. We exclude LLM recommenders without a self-improvement loop, conversational recommendation in general, and general hyperparameter-optimisation libraries. We also exclude methods that use an LLM once as a fixed training signal for a conventional recommender (such as offline LLM pairwise supervision applied only where a gate on history length, item popularity and model uncertainty predicts it is reliable [155]), and methods that evolve the recommendations rather than the recommender (such as QD-Rec's quality–diversity archive of items and hypotheses [65]). Finally, we exclude continual-learning methods that call themselves self-evolving but have no proposer or selection step, such as LION's sparse key–value memory for a generative recommender [100]. Seven studies outside recommendation are included as comparison points because they inform evaluation: learning to rank [136], search ranking [182], e-commerce search strategy [173], lexical retrieval [130], generative document retrieval [129], lifetime-value (LTV) prediction [179], and a controlled study of agent feedback on public CTR and learning-to-rank (LTR) data [196].

Search protocol. The cut-off date is 22 October 2026. We started from keyword searches over arXiv, OpenAlex and Semantic Scholar ("self-evolving recommender", "LLM agent recommender model optimisation", "agentic harness recommender", "LLM-driven architecture search CTR" and survey queries). We then walked the reference lists of the 2026 systems (RecHarness, EvoRec, AgentX-Model, RecEvolve) and of the agentic-recommender roadmap backwards and their titles forwards, and finished with about thirty full-text arXiv phrasings combined with company names. Generic keyword search returned mostly unrelated "agent for *X*" papers (§9). This first sweep yielded 22 of the audited systems and five comparison points. A second sweep then ran 190 further arXiv queries (817 new candidates, all screened by title; the last 31 repeat the company-name search on abstracts, logged query by query). It also enumerated 13 ACM proceedings volumes through OpenAlex (RecSys, KDD, SIGIR and WWW 2025–2026 including the WWW

Table 1: Flow of records. Counts of the first sweep are of systems, not records, because it followed citation chains rather than enumerating a database.

First sweep: keyword search and citation chains; recommender loops coded	22
comparison points coded	5
Second sweep: arXiv candidates dated 2025+, screened by title (190 queries; §2)	817
ACM proceedings records, title keyword filter (13 volumes)	6,874
keyword hits screened by title	536
candidates read in full or in abstract	9
entered as core loops / comparison points	5 / 2
known only from abstracts, not coded	2
Read in full and excluded as borderline	4
Recognised too late for the double coding, catalogued but not yet coded (three found by the first sweep as adjacent work, one already cited)	4
Audited: core loops / comparison points	27 / 7
In-scope loops found: 27 core, 6 not yet coded, 1 blog post only	34

Table 2: Audit codebook. C7 is recorded as free text and discussed in F8; the other eight criteria are scored (values in Table 6).

C1	Metric: full-catalogue ranking, sampled ranking (with N negatives), AUC/loss, or online only
C2	Selection signal: which split or signal accepts or rejects each candidate
C3	Test discipline: is the selected candidate evaluated on a held-out test once, after search? (No if test data are used during the search or repeatedly, even if the scores are not used to select.)
C4	Variance: repeated seeds or repeated evolution runs for headline numbers
C5	Overfitting check: validation-to-test gap or held-out transfer of the selected candidate
C6	Null control: random search, HPO or another non-LLM search at an equal or larger trial budget (number of candidates evaluated), LLM cost not required to match (the coder guide asked for the same space; F6 gives the stricter counts)
C7	Budget: what is counted (iterations, GPU time, tokens, money)
C8	Baselines: are starting points and compared models tuned?
C9	Isolation: can the agent reach test data or evaluation code? Reported incidents of gaming or leakage are recorded separately.

companions and the February KDD 2026 cycle, CIKM 2025 and WSDM 2026; 6,874 records filtered by a title keyword pattern, and the 536 hits screened by title), followed forward citations of 16 seed systems in Semantic Scholar, and searched industry engineering blogs. It found nine further candidates: five entered the audit as core systems, two as comparison points, and two are known only from their abstracts (§6). CIKM 2026, DBLP and OpenReview could not be searched automatically.[1] Titles were screened by a single screener with LLM assistance (for the ACM volumes, after a keyword filter), not in duplicate. The recall of the screen is therefore not measured, and individual title decisions were not logged.[2] Only hits dated 2025 or later were screened, so the second sweep cannot have found earlier systems; Appendix B gives the per-query hit counts and the bibliography checks. Table 1 counts systems for the first sweep, which was logged as the systems it found, and records only for the second.

Audit procedure. We coded nine criteria (Table 2), eight of them scored and C7 as free text, for each configuration- or code-level system, in six steps. (1) Extraction: LLM agents read the full text (arXiv HTML or PDF, including appendices) and extracted a quote for each cell. (2) First coding: the first author coded each cell from that quote and its context, checked against the paper text but without a full reading of every paper. (3) Verification: LLM verification passes, reviewed by the authors, re-checked the cells and the cited sentences. (4) Second coding: a second author coded every core system by hand from the full text (without an LLM, by the coder's own account), not independently. (5) Reconciliation: LLM agents drafted a resolution for each disagreement, which the authors reviewed. (6) Online gates: one author coded the online-experiment items of F9, and a blind LLM re-coding checked them. Table 11 and §9 say who coded what, and Appendix B gives the full history. We audit what the papers report: a criterion is coded "not stated" when the paper does not say, which is different from "no". Each "yes" or "partial" cell is backed by a verbatim quote of at most 25 words with a section pointer. Since no quote can show an absence, most "no" and "not stated" cells give the section searched or a short reason instead. The data file and the evidence are released[3] so that the coding can be checked and corrected. For systems whose launch decision is an online experiment, we read the offline criteria as describing the offline screen, and assess the launch evidence separately (R9 in §7).

3 Background

Related surveys. Surveys of LLMs for recommendation treat the LLM as a feature encoder, a ranker or a generator [96, 183], and surveys of LLM agents for recommendation treat the agent as the recommender, a conversational interface or a user simulator [63, 138, 218]. Surveys of self-evolving agents treat the agent's own prompts, memory, tools or weights as the object that evolves, and mention recommendation at most in passing (a two-sentence mention of self-optimised fine-tuning and a recommendation task in a benchmark it describes in [47], none in [40]). The most recent roadmap for agentic recommender systems mentions the "agent as engineer" direction in one clause of its task list, which cites AgentX and the Google loop [99]. The most recent peer-reviewed survey of agentic recommenders organises 47 studies by where the agent sits in the serving workflow, not by what changes the recommender [188]; we judged it from its abstract and resolved reference list, since we could not obtain the full text. A general survey of how automated research is evaluated covers no recommender system [211]. Closest to our audit, a concurrent controlled study shows on CTR and learning-to-rank data that exact validation feedback lets a coding agent overfit the validation split, and that a feedback contract with a quarantined test set removes the

[1] OpenReview was covered only by manual web searches, and its PDFs were blocked.
[2] The keyword filter, every second-sweep query with its hit count and the per-volume counts are released with the search log, but the list of candidate identifiers was not saved, so the screen can be re-run from the logged queries but not checked record by record (arXiv's index has changed since).
[3] https://github.com/shangliy/recforge/tree/master/papers/self-evolving-recsys-survey/audit

spurious gain on a task with no signal [196]. Beel et al. call for autonomous recommender-systems research labs and for benchmarks to evaluate them [9]. RMD-Bench [137] fixes protocols for cross-dataset and cross-scale discovery and ExecML-HSTU [28] measures whether agents' edits execute correctly, but no shared benchmark yet compares evolvers under a sealed test, full-ranking evaluation and equal-budget null controls.

Search-space AutoML for recommenders. Automated machine learning (AutoML) for recommenders searched for feature-interaction structures (AutoCTR [159], AutoFIS [104]), embedding dimensions (AutoEmb [220], AutoDim [221]), whole architectures with a weight-sharing supernet (NASRec [215]), compact attention-based sequential models under resource constraints (DNS-Rec [214]), embedding tables (OptEmbed [114]) and loss functions (AutoLossGen [93]); two surveys cover this line [21, 223]. Each method can only return a design that its space contains. Evaluation is mostly validation logloss or AUC on public CTR data, or standard rating and sequential benchmarks, with standard splits. Even in this setting, careful re-evaluation of architecture search found that the search phase often added little over random sampling and that the evaluation protocol dominated reported differences [89, 190, 200]. The community answered with tabular and surrogate benchmarks that make equal-budget comparisons cheap [36, 39, 198] and with a best-practice checklist [101]. The LLM loops below inherit these problems with a larger, open-ended space and with control over the evaluation code.

LLM program evolution and ML-engineering agents. FunSearch [150] and AlphaEvolve [131] pair an LLM that proposes code with an automatic evaluator and an archive of programs, a pattern that AutoML-Zero had applied to whole learning algorithms with random mutation instead of an LLM [142]. Open systems followed: OpenEvolve [132], ShinkaEvolve [83] and DeepEvolve [107]. LLM-guided evolution was applied earlier to neural architecture search (NAS) (EvoPrompting [20], LLMatic [126], GPT-NAS [199], and GPT-4 as a NAS proposer [222]), to heuristics [197], reward functions [115] and scientific equations [156], and to prompts and LLM pipelines as optimisation targets (OPRO [191], EvoPrompt [50], DSPy [76], TextGrad [202]); the last are the general form of the L4 prompt loops of §D.5. ML-engineering agents—AIDE [69], MLE-STAR [124], R&D-Agent [195], AIRA [167], DS-Agent [51], AutoML-Agent [168], Agent Laboratory [152] and the AI Scientist [112, 189]—apply the same pattern to whole training pipelines, and recent work optimises the agent harness itself [86, 111, 178, 187]. Their benchmarks contain at most one recommendation task: MLE-bench includes the H&M fashion-recommendation competition [17] and MLRC-Bench a next-product-recommendation task [217]. MLRC-Bench shows agents only the development split and scores the best development snapshot once on test, whereas MLGym's `validate` command returns test-set metrics during the session [127]. Broader benchmarks for data-science and research agents [23, 26, 70, 81, 140, 161] contain no recommender-model task, and an independent evaluation of an AI-scientist system found its claimed results hard to reproduce [10].

Generative recommenders. HSTU [205] and TIGER [141] recast recommendation as sequence generation over item IDs or semantic IDs, and OneRec unifies retrieval and ranking in one generative model that is further aligned on its own samples, first by iterative direct preference optimisation (DPO) [34] and then by on-policy reinforcement learning (RL) [225, 226]. These models add new objects that a loop can evolve: tokenisers, codebooks, decoding procedures and generative losses.

Evaluation methodology. Tuned simple baselines beat many published neural recommenders [32, 147]; a loss change alone can reorder SASRec [72] and BERT4Rec [79, 162], and BERT4Rec's reported advantage depends on much longer training [139]; sampled metrics are inconsistent with full-ranking metrics and can reverse model rankings [57, 82, 145]; leave-one-out splits that ignore the global timeline leak future information [57, 67], and the split choice changes rankings [120]. Systematic re-examinations found that most neural recommenders could not be reproduced or were beaten by tuned simple baselines [31], that evaluation choices varied widely across papers [164], and that with untuned baselines any proposed model can win [153]. Libraries and open benchmarks such as RecBole [219], Elliot [3], DaisyRec [163], BARS [228] and FuxiCTR [229] standardise protocols for human-designed models. For adaptive reuse of a holdout, theory predicts overfitting [13, 38]; empirically, a meta-analysis of 120 Kaggle competitions found little [149]. For LLM loops specifically, keep-if-better acceptance is selection under noise, with selection-set scores well above held-out accuracy when the selection set is small [61], and best-of-N runs produce apparent gains from selection alone [49]. For LLM research agents, the scarcity of overfitting has been tied to how compressible their successful strategies are [12], and AI-scientist systems hide post-hoc selection, leakage and metric misuse [113]. Beyond recommendation, results depend on seeds and on the budget spent tuning them [14, 35, 56], leakage is a recurring cause of irreproducible ML-based science [74], and agent benchmarks are sensitive to cost, budget and task-setup flaws [64, 75, 177, 230]. For online selection, the relevant tools are variance reduction with pre-experiment data [33], interleaving for more sensitive ranking comparisons [19], sequential tests that stay valid under repeated looks [71], and off-policy replay and doubly robust estimation for evaluating policies on logged data before an experiment [37, 88]. We return to this tension in §6. Figure 2 places these precursors next to the 2026 systems.

4 A Taxonomy

A recent blueprint for self-evolving agentic recommenders distinguishes reinforcement-learning evolution in defined action spaces from LLM generation and selection in open design spaces [59]. We describe a loop by three separable choices: what it changes, who proposes the change, and which signals decide whether the change survives (Table 4). Separating the proposer from the object avoids a conflation that is easy to make, in which "AutoML" names a kind of proposer (search over a written-down space) but is listed as if it were a kind of object.

Axis 1: the evolved object. **L1 configuration**: values in an existing configuration—hyperparameters, fusion weights, retrieval budgets, thresholds, strategy parameters. **L2 code**: executable model, loss, training or feature code, however small the edit. **L3 generative**

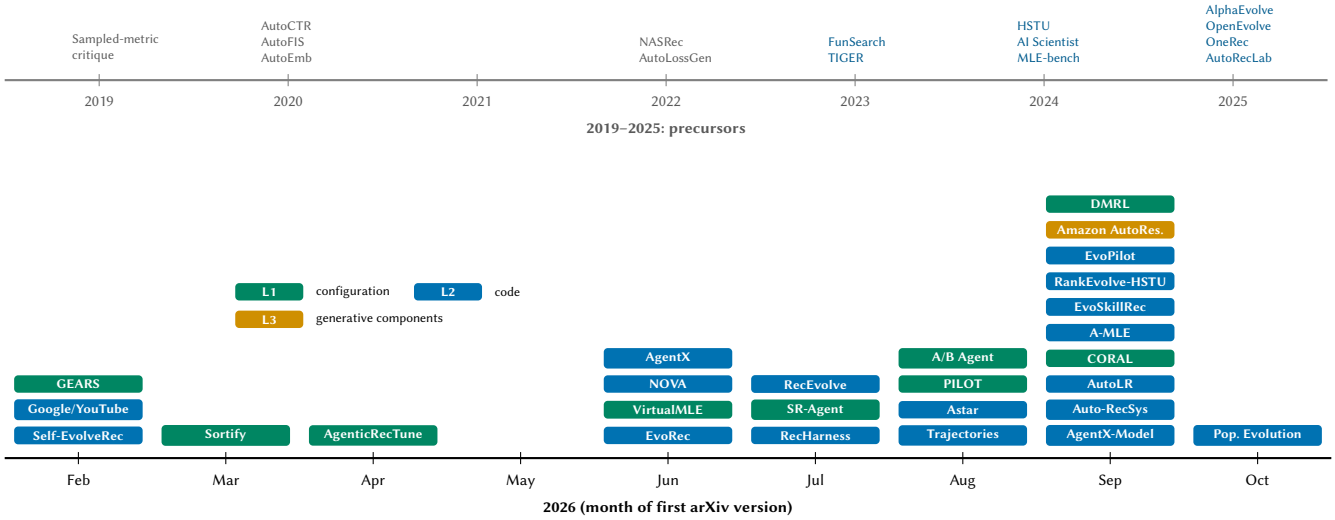


Figure 2: From search-space AutoML to agent-driven evolution. Top: precursors by year (grey: search-space AutoML and evaluation critique; blue: program evolution, ML-engineering agents and generative recommenders). Bottom: the 2026 configuration-, code- and component-level systems surveyed here, by the month of their first arXiv version (RecEvolve's v1 was submitted on 20 July, per its arXiv page, although it was announced under a September identifier); the colour gives the highest level of the evolved object. Only the coded recommendation loops are shown; the comparison points outside recommendation, the surveyed but uncoded systems and the L4 systems are omitted.

components: semantic-ID construction, item tokenisers and vocabularies, and decoding of a generative recommender; L3 takes precedence when such a component is changed through configuration or code. **L4 agent policy**: the prompts, memory, skills or weights of a recommender that is itself an LLM agent. A loop that changes several objects carries several labels (L1+L2 when an agent edits both learning rates and model code); 10 of the 27 audited loops do.

Axis 2: the proposer. An *enumerated space*: classical search (grid, Bayesian optimisation, evolutionary or differentiable architecture search) over options a person wrote down; this is search-space AutoML. An *LLM*: a language model writes the change, free-form or inside human-defined edit arms. A *hybrid*: a classical selector chooses the direction and an LLM writes the change (RecHarness's Thompson sampling over edit arms [102]), or an LLM steers a classical search (Sortify's controller around Optuna [30]). A *learned proposer*: a model trained on the system's own history proposes the next direction (Astar [60]; we also count DMRL here, whose RL-trained LLM writes the skill document a frozen LLM agent follows, coded `learned+llm` in the data). Orthogonal to these, a loop is *meta*-evolving when it changes the proposer's own instructions, tools, skills or weights and reuses them later; keeping an archive or a log of past results does not count. By this rule 11 of the 27 audited loops are meta-evolving (Table 5). We flag separately a loop whose *evaluator* evolves, that is, whose feedback code, objective or reward model changes during the run: Self-EvolveRec's diagnosis tool co-evolves with the model, Sortify recalibrates its search objective each round, and Astar recalibrates its reward model. An evolving evaluator is a selection problem of its own, because the measure that accepts candidates changes with them. The first two axes are descriptive; Table 3 relates them to the audit outcomes. The object of evolution coincides with the stratum: eight of the nine configuration-only loops select online, and 15 of the 18 loops that edit code or models select offline, which is where the offline criteria bite. Proposer type and meta-evolution show no clear pattern in the offline criteria; the groups are small, and the table describes the corpus rather than estimating an effect. All four reported incidents come from loops that edit code, which with four incidents is an observation, not a rate.

Table 3: Taxonomy against audit outcomes (core loops; groups as small as $n = 2$ and $n = 6$ are listed for completeness, and their shares should not be compared). Offline: share of loops that select offline; the next two columns, a named validation split (Val) and test used once (Test), are among those loops. Null: a non-LLM null run in full or in part (C6 Y or P); Isol.: C9 Y or P; Incid.: a gaming or leakage incident reported. Cells are counts over the group, so small groups describe, not estimate.

Group	Offline	Val	Test	Null (Y/P)	Isol. (Y/P)	Incid.
All core ($n = 27$)	16/27	4/16	2/16	8/27	23/27	4/27
LLM proposer ($n = 19$)	12/19	2/12	1/12	6/19	16/19	4/19
Hybrid proposer ($n = 6$)	3/6	2/3	1/3	2/6	6/6	0/6
Learned proposer ($n = 2$)	1/2	0/1	0/1	0/2	1/2	0/2
Meta-evolving ($n = 11$)	6/11	2/6	1/6	5/11	9/11	0/11
Not meta ($n = 16$)	10/16	2/10	1/10	3/16	14/16	4/16
Config only (L1) ($n = 9$)	1/9	1/1	1/1	1/9	9/9	0/9
Code or model (L2+) ($n = 18$)	15/18	3/15	1/15	7/18	14/18	4/18

Axis 3: the selection chain. A candidate passes an ordered chain of signals, one of which decides inside the loop. **Self**: an LLM critic, council, auditor or simulator judges the candidate before or instead of measurement. **Proxy**: a reduced-scale screen, a proxy model, a training-loss curve or a replay. **Offline**: evaluation of the trained candidate on held-out data. **Online**: an A/B experiment. Industrial loops typically chain several, for example self→offline→online; whether the offline split is a validation or a test split, and whether it is stated, is a property of the evaluation that we audit (§6), not of the design.

Each signal fails in a characteristic way, and the failures are of two kinds [116]. *Regressional* failures come from selecting the maximum of noisy estimates: an offline split reused over hundreds of candidates rewards noise [38], a proxy ranks candidates differently from the full model, an online round selected as best overstates its effect [85], and an LLM judge from the same model family as the proposer shares its errors. *Adversarial* failures come from an optimiser that changes what is measured: an agent that can edit the evaluation code, the batch size of an in-batch metric or the evaluation date can raise the score without improving the recommender [157, 224]. The audit checks the first kind with C2–C6 and the second with C9.

5 What the Systems Share

Appendix D describes each system, level by level; this section records what the systems share and what their loops kept.

5.1 Recurring designs

Table 5 places the 27 audited loops on the three axes and adds the role of humans; the released `design.csv` also records each loop's memory, acceptance rule, guards and LLM, with evidence. Four patterns recur.

Where the chain ends. Eight loops end at an offline evaluation: three open harnesses (Self-EvolveRec, VirtualMLE, EvoSkillRec) and five industrial research loops whose papers report offline results, or in Auto-RecSys's case only operational ones (A-MLE, Auto-RecSys, RankEvolve-HSTU, Population Evolution, Amazon AutoResearch). The other 19 pass candidates to an online experiment, and 14 of them involve humans: eleven approve launches, rounds or candidates, and three check or intervene. The two ends are exposed to different failures. A loop that ends offline controls the code and data that produce its final number, so the adversarial failures matter most; a loop that ends online cannot change the measurement easily but selects on expensive, noisy experiments, so the regressional ones do.

LLM review before measurement. Eight loops make an LLM critic, council or auditor a gate that every candidate must pass before it is measured (Table 5, chains starting with S); others, such as RecEvolve, A-MLE and RankEvolve-HSTU, use LLM critics to rank or filter ideas. These gates are cheap and catch invalid edits—NOVA's verifier rejects target leakage [109]—but they add a selection stage whose judge often comes from the same model family as the proposer, the condition under which a self-generated signal is least reliable [48, 193].

Hybrids that keep a classical selector. Six loops leave the choice of direction or the numeric search to a classical method and use the LLM to write or steer: Thompson sampling over edit arms (RecHarness), an evidence-weighted selector (AutoLR), typed operators over a skill library (EvoSkillRec), Optuna (Sortify), Pareto enumeration (GEARS) and a deterministic action enumerator (PILOT). Because the classical part already defines a space, these designs could run an equal-budget null over it with little extra code; only EvoSkillRec reports one (against NASRec), and RecHarness's random arm router still applies LLM-written edits.

Humans in the loop. Only nine papers describe no human role in the loop. Five have a person approve or review each round or candidate (AgentX, AgenticRecTune, A-MLE, EvoPilot, A/B Agent). Under the rule of §2 this does not remove them from scope, because the person approves or vetoes what an evaluation kept. It does mean that their reported gains are those of a human–agent system.

5.2 What the loops kept

Three papers report no kept edit (Auto-RecSys, AutoLR, DMRL), Astar reports only proposals in case studies, and A-MLE only technique families. The other 22 name the changes their loops kept or launched, and we coded the 87 edit descriptions they give by kind (a description that groups several updates, such as SR-Agent's ten, counts once) (`design.csv`; the tally is generated by `summarize.py`). The eight configuration-only loops that name kept edits (DMRL, the ninth, names none) kept strategy parameters (17 of their 20 edits): fusion weights, sorting-formula coefficients, retrieval budgets, thresholds. The fourteen code-level loops kept most often architectural changes (29 of 67) and loss or objective changes (12), then optimiser and schedule settings (8), data and negative sampling (6), and a few strategy, regularisation, capacity, feature and other changes. As far as the papers report their kept edits (which they chose to describe, and which one author coded), we hypothesise that code-level loops do more than expensive hyperparameter optimisation (HPO). R5's null would test this hypothesis, and it matters for the choice of null control (R5 in §7): an equal-budget HPO run over the changed knobs is a necessary but not a sufficient null for them, and R5's recommended null for a code-level loop mutates its code at random.

Many of the named edits are established techniques transplanted into a particular model: InfoNCE losses and hard negatives [18, 78, 123], DCN-V2 layers and cosine learning-rate decay [135], SwiGLU feed-forward blocks [16, 109], gated paths [170], RMSNorm [231], expert pools [194] and checkpoint averaging [137]. This is useful engineering, and finding which known technique helps a given production model is much of what human model iteration does. It also suggests a null that no paper runs: a scripted sweep over a catalogue of known techniques at the same budget. One caveat applies: papers report the edits they find interesting, not all kept edits.

6 How Are Self-Evolving Recommenders Evaluated? An Audit

Audited set. We audited the 27 recommendation loops at levels L1–L3 that keep or reject candidates by evaluation, and the seven comparison points of §2, in two strata: 16 *offline-selecting* loops, which decide on offline or proxy evaluations (some later confirm

Table 4: Taxonomy: evolved object (rows) × proposer (columns). Multi-object loops appear in the row of their highest level. † open-source code verified; [m] meta-evolving (the loop also changes its proposer); [e] the loop's evaluator evolves; [u] not yet audited. Selection chains of the audited systems are in Table 5.

	Enumerated space	LLM	Hybrid (search + LLM)	Learned proposer
L1 configuration	AutoEmb [220], AutoDim [221]	VirtualMLE[†m] [16], CORAL [5], AgenticRecTune[m] [184], SR-Agent [192], A/B Agent [68]; *comparison:* PEAR [173]	Sortify[e] [30], GEARS [203], PILOT[m] [97]	DMRL[m] [216]
L2 code	AutoCTR[†] [159], AutoFIS[†] [104], NASRec[†] [215], DNS-Rec [214], AutoLossGen[†] [93]	Self-EvolveRec[†e] [78], Google/YouTube [170], EvoRec[m] [123], NOVA[m] [109], AgentX[m] [84], AgentX-Model[m] [194], Auto-RecSys[m] [90], RecEvolve [135], A-MLE [46], RankEvolve-HSTU [28], Population Evolution [137], Trajectories-to-Evidence [231], EvoPilot [42], Meta REA (blog) [121], AutoModel[u] [208], RuleAgent[u] [176]; *comparison:* EvoRank[†] [136], AI Co-Scientist [182], RankEvolve-IR[†] [130], AgentLTV[†] [179], Feedback Contract [196]	RecHarness[†] [102], AutoLR [213], EvoSkillRec[†m] [92]	Astar[me] [60]
L3 components		Amazon AutoResearch [18], AgenticTagger[u] [185], Agent0[u] (feature prompts) [158]; *comparison:* Evolving Semantic Indexes [129]		
L4 agent policy	Selected mostly on self-generated feedback. Memory and reflection: AgentCF[†] [209], AgentCF++[†] [108], SAGER [165], RecNet [87], AtomRec [62], MemRec[†] [25], Re2LLM [174], RecPilot [133]; weights: OneRec [34], SPRec[†] [43], Rec-R1[†] [98], CoARS [175], RSIR[†] [210], EvoReason [232]; prompts and tool descriptions: RecPrompt [105], Spotify [134]			

Table 5: Design of the 27 audited loops: the object evolved, the proposer, the chain of selection stages, whether the loop evolves itself, and the role of humans. Evidence for every cell is in the released `design-evidence.md`.

System	Object	Proposer	Chain	Meta	Humans	System	Object	Proposer	Chain	Meta	Humans
Self-EvolveRec	L2	LLM	**O**	N [e]	none	Sortify	L1	hybrid	P**A**A	N [e]	launch
RecHarness	L1+L2	hybrid	**O**A	N	none	GEARS	L1	hybrid	**A** (logged)	N	launch
Google/YouTube	L1+L2	LLM	O**A**	N	launch	RecEvolve	L2	LLM	P**O**A	N	checkpoints
EvoRec	L1+L2	LLM	**O**A	Y	none	Astar	L2	learned	P**O**A	Y [e]	none
VirtualMLE	L1	LLM	**O**	Y	none	A-MLE	L1+L2	LLM	**O**	N	each
NOVA	L1+L2	LLM	S**O**A	Y	checkpoints	SR-Agent	L1	LLM	SP**A**	N	launch
AgentX	L1+L2	LLM	S/O**A**	Y	each	EvoSkillRec	L2	hybrid	**O**	Y	none
AgentX-Model	L2	LLM	S**O**A	Y	launch	RankEvolve-HSTU	L1+L2	LLM	**O**	N	checkpoints
Auto-RecSys	L2	LLM	**O**	Y	checkpoints	Population Evol.	L2	LLM	**OPO**	N	none
AutoLR	L1+L2	hybrid	S**O**A	N	launch	Trajectories	L2	LLM	S**O**A	N	none
CORAL	L1	LLM	**A**	N	checkpoints	EvoPilot	L1+L2	LLM	**O**A	N	each
AgenticRecTune	L1	LLM	S**A**	Y	each	Amazon AutoRes.	L1–L3	LLM	**O**	N	checkpoints
						PILOT	L1	hybrid	P**A**A	Y	launch
						A/B Agent	L1	LLM	S**A**	N	each
						DMRL	L1	learned	A**P**	Y	none

Chain: S self (LLM critic, council or simulator), P proxy or screen, O offline evaluation, A online experiment; the stage in bold decides inside the loop. Meta: the loop changes its own proposer; [e] its evaluator also or instead evolves. Humans: approve each round or candidate (each), approve launches (launch), check at stage boundaries or intervene (checkpoints), or none stated; where a loop has more than one role, the strongest is shown (A-MLE also checks at stage boundaries); humans who only design the loop, its objective or its arms, as in RecHarness, are not counted.

Strata: Google's offline stage ranks candidates by validation loss and its online stage decides which run and launch; Sortify's proxy search proposes each round and the rounds are chosen online; DMRL's deciding proxy is a long-term predictor applied to online experiment outcomes, so the audit counts it as online-selecting. GEARS's segment-level estimates likewise come from the logged outcomes of a completed randomised experiment, so it too is counted as online-selecting. Because its agent makes one guided selection pass per experiment rather than iterating, it is a borderline case of the 'repeatedly' in our definition, and without it the online-selecting stratum is nine loops, of which none corrects across rounds and one confirms afresh. CORAL and DMRL have no explicit keep/reject rule but are counted under the codable rule because an online experiment's outcome decides what survives: CORAL deploys each cycle's configuration to an A/B test and reverses the changes that did not help, and DMRL's optimizer is updated towards the skill-document edits whose online experiments score higher, a soft selection by experiment outcome. Without GEARS, CORAL and DMRL, the stratum is seven loops, of which none corrects across rounds (one partially) and one confirms afresh (two partially).

online), and 10 *online-selecting* loops, which decide on A/B experiments and are assessed on the design of those experiments. The remaining loop, Auto-RecSys, reports only operational outcomes and makes no accuracy claim. Six in-scope systems are not yet coded: four were recognised only after the second coding was returned, and two are known only from abstracts. Appendix B names

them and the borderline systems read but not counted. Every count below is about the 27 coded loops. Table 6 gives the coding, Figure 3 summarises it, and Table 7 states each headline count once, with its denominator, range, coder, agreement and how far it could move if the six were added. Every coded count is recomputed by script from the released data, but a few judgement-based sets are listed in the script rather than stored as data columns.[4]

How to read the counts. We state the limits of every count here once, and do not repeat them. (i) A count describes the coded corpus, not the field, and the corpus comes from a citation-led search whose recall was not measured (§9). (ii) It describes what the papers report: "not stated" is not "no", so where a count rests on such cells we give the range it would span if each were read either way. (iii) It rests in effect on one author's reading. Two authors coded the 27 loops, but not independently, and the released codes follow the second coder's sheet in 236 of 243 cells; no rater outside the team was involved. We therefore tag F1–F3, F7 and F9 *provisional*, and F5, F6 and F8, which rest on low agreement or on one coder, *descriptive*: we report their evidence and draw no conclusion from the counts. F4's zero needs no judgement. Table 7 gives each count with its denominator, range, coder and agreement, and how far it could move if the six uncoded loops were added. Every credited cell is quoted in the released evidence, and a blind re-coding of the 48 cells behind F2–F4 by an outside rater is the first check we invite. Figure 3 summarises the coding, and the findings follow.

F1 (provisional): Few loops state that they rank the full catalogue. Of the ten offline-selecting loops that report ranking metrics, three rank the full catalogue, three use sampled candidates and four do not say, so between three and seven use full ranking. The three full-ranking loops are RankEvolve-HSTU, Population Evolution and the retrieval system of Amazon's AutoResearch. Self-EvolveRec and RecHarness pair each test item with 99 sampled negatives, and we code RecEvolve as sampled. Amazon's agent also changed the candidate set during the campaign: a text-only fallback "expanded catalog coverage by 5.8×" [18], so its earlier evaluations ranked a smaller candidate set than its later ones. Five offline-selecting loops evaluate ranking-stage CTR models with AUC or a loss, as is standard for that stage (§9), and AutoLR selects offline but headlines only online lifts.

F2 (provisional): Half the offline loops do not state their selection data. Of the 16 offline-selecting loops, eight do not say which data select candidates, four select on the data they report, and four name a validation split, so between 4 and 12 select on the data they report (Table 7). The four naming validation are Self-EvolveRec, RecHarness, VirtualMLE and EvoSkillRec. Of the four that select on the data they report, RankEvolve-HSTU's persistent leaderboard is the canonical full-test NDCG@10 [28], and Amazon's AutoResearch "retains modifications that improve a held-out scalar metric" and headlines the trajectory's peak [18]. EvoPilot selects and reports

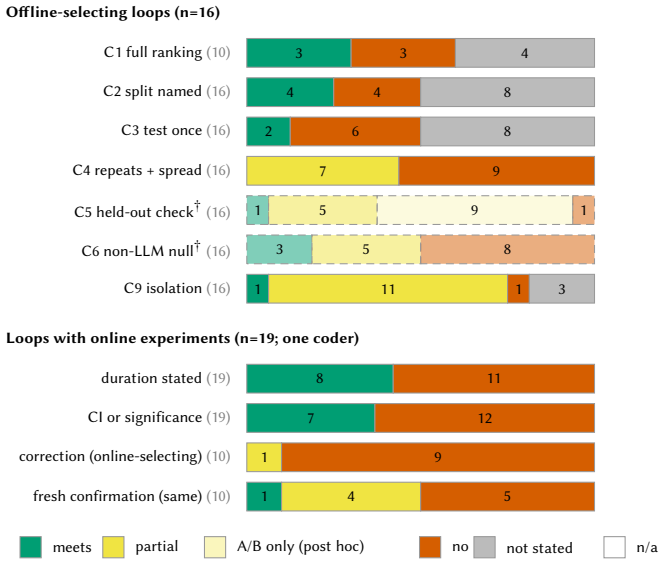


Figure 3: Audit summary, generated from the released `audit.csv`. Top: the 16 offline-selecting loops (C1 among the 10 that report ranking metrics; sampled ranking is shown as no; nine of the 14 partial C5 codes are online confirmations, drawn separately because they test transfer rather than overfitting to the selection data). C5 and C6, whose agreement is lowest, are drawn faded and marked †. C9 counts the isolation design; incidents are F7. Bottom: the design of the online experiments reported by 19 loops, coded by one author; correction and confirmation are counted over the ten loops that select online. Reliability of each bar: Table 7.

on one held-out replay set [42]. Population Evolution selects on its source instances' evaluation scores and reports its local gains on them, while withholding the transfer targets from the proposer; it also calibrates the shared baseline configuration on the released test split [137]. All four disclose these choices, which is to their credit. The eight silent loops are EvoRec, NOVA, AgentX-Model, AutoLR, RecEvolve, Astar, A-MLE and Trajectories-to-Evidence. In all eight the keep decision is an offline score on unnamed data, and all but A-MLE then A/B-test the kept candidate as a confirmation.[5] Outside recommendation, the travel-search co-scientist keeps or reverts each change on the same held-out metric that it reports [182].

F3 (provisional): Two of 16 offline loops state test-once use. Of the 16 offline-selecting loops, two state that the selected candidate was evaluated on held-out test data once after the search (RecHarness, VirtualMLE), six use the test data during the search or repeatedly, and eight do not say, so between 2 and 10 are test-once. Of the six, four report on the data they select on (RankEvolve-HSTU, Population Evolution, Amazon, EvoPilot), so their F3 code follows from their F2 code and is not separate evidence. Only two of the six

[4]The audit, crosstab, sensitivity and proxy-check tables and the figure are generated from them by `summarize.py`; the value columns of the headline table, Table 7, are printed by `summarize.py --headline`, which names the judgement-based sets (agent-introduced incidents, stated catchers, the same-space and strict null controls) and reads the F8 lift range from `f8-lifts.md`; the design table, Table 5, was assembled by hand from the data files (the kept-edit tally is generated by `summarize.py` from hand-coded fields).

[5]The score is judged in AgentX-Model by an LLM, in A-MLE by a significance test against a rolling baseline followed by human review, and in Trajectories-to-Evidence together with an LLM-assisted claim check.

Table 6: Evaluation audit (full-text versions as listed in the released audit file; search cut-off 22 October 2026). Level: evolved object. C1 metric: Full = full-catalogue ranking; Sampled = ranking against sampled negatives; Rank? = ranking metric with the candidate set not stated; AUC = AUC/logloss/loss proxy; A/B = online only; other = a non-recommendation metric. C2 selection signal: val, rep. (the data that are also reported, test or held-out; `test` in the data file), A/B, NS. C4 P: repeats without spread, repeats in an ablation only, a re-run rule, or a noise estimate of single models. C5 P: partial transfer evidence; O: only an online confirmation of the kept candidate, which tests transfer rather than overfitting to the selection data (credited partial in the data file; Appendix B). C6 Y: equal or larger trial budget (LLM cost need not match; F6); P: unmatched trial budget, LLM still proposing, or replay only. C8: Y tuned, N untuned, P partly or on test data, prod. = production model, pub. = published numbers, seed = the loop's seed program. C9 isolation design: P part of the loop constrained (fixed evaluation code, gates, whitelists, sandboxing, bounded actions); RankEvolve-HSTU is N because its read-only evaluator is used only in its ExecML benchmark, not in the recommender loop, whose agents could change the evaluation; ^{G} = the paper reports gaming, metric-winning bugs, metric inflation or leakage by the loop. Code: later = promised. Y = yes, N = no, P = partial, NS = not stated, – = not applicable. Reliability of each column: Table 7. For the online-selecting loops, C1–C6 describe the offline screen, if any, so a N there is not a failing of the launch evidence, which F9 assesses.

System	Level	C1 metric	C2 select	C3 test once	C4 repeats	C5 gap	C6 null	C8 baselines	C9 isolation	A/B	Code
Self-EvolveRec [78]	L2	Sampled	val	NS	N	P	P	N	NS	N	Y
RecHarness [102]	L1+L2	Sampled	val	Y	P	O	P	seed	P	Y	Y
Google/YouTube [170]	L1+L2	A/B	A/B	–	P	P	N	prod.	P	Y	N
EvoRec [123]	L1+L2	Rank?	NS	NS	N	O	P	N	NS	Y	N
VirtualMLE [16]	L1	Rank?	val	Y	P	P	Y	Y	Y	N	Y
NOVA [109]	L1+L2	AUC	NS	NS	N	O	Y	prod.	P	Y	N
AgentX [84]	L1+L2	A/B	A/B	–	N	N	N	prod.	P	Y	N
AgentX-Model [194]	L2	AUC	NS	N	P	O	P	prod.	P	Y	N
Auto-RecSys [90]	L2	other	–	–	N	N	N	prod.	P	N	N
AutoLR [213]	L1+L2	A/B	NS	NS	N	O	N	prod.	P	Y	N
CORAL [5]	L1	A/B	A/B	–	N	N	N	prod.	P	Y	N
AgenticRecTune [184]	L1	A/B	A/B	–	N	N	N	prod.	P	Y	N
Sortify [30]	L1	A/B	A/B	–	N	P	N	prod.	P	Y	N
GEARS [203]	L1	other	A/B	–	N	P	N	prod.	P	Y	N
RecEvolve [135]	L2	Sampled	NS	NS	N	O	N	prod.	P ^{G}	Y	N
Astar [60]	L2	Rank?	NS	NS	N	O	N	prod.	NS	Y	N
A-MLE [46]	L1+L2	AUC	NS	NS	P	N	N	prod.	P	N	N
SR-Agent [192]	L1	A/B	A/B	–	N	P	N	prod.	P	Y	N
EvoSkillRec [92]	L2	AUC	val	N	N	Y	Y	N	P	N	Y
RankEvolve-HSTU [28]	L1+L2	Full	rep.	N	N	P	N	pub.	N ^{G}	N	later
Population Evolution [137]	L2	Full	rep.	N	P	P	P	P	P	N	later
Trajectories-to-Evidence [231]	L2	AUC	NS	NS	N	O	N	prod.	P	Y	N
EvoPilot [42]	L1+L2	Rank?	rep.	N	P	O	N	prod.	P ^{G}	Y	N
Amazon AutoResearch [18]	L1+L2+L3	Full	rep.	N	P	P	N	Y	P ^{G}	N	N
PILOT [97]	L1	A/B	A/B	–	N	N	N	prod.	P	Y	N
A/B Agent [68]	L1	A/B	A/B	–	N	N	N	prod.	P	Y	N
DMRL [216]	L1	A/B	A/B	–	N	N	N	prod.	P	Y	N
Comparison points outside recommendation											
EvoRank (LTR) [136]	L2	other	val	Y	Y	Y	Y	Y	Y	N	Y
AI Co-Scientist (search) [182]	L2	AUC	rep.	N	N	N	N	P	NS ^{G}	N	N
RankEvolve-IR (lexical) [130]	L2	Full	NS	Y	N	Y	P	N	N	N	Y
Evolving Semantic Indexes (GR) [129]	L3	Full	val	Y	P	Y	N	seed	Y	N	N
AgentLTV (LTV) [179]	L2	other	rep.	N	N	N	P	NS	N	N	Y
Feedback Contract (CTR/LTR) [196]	L2	AUC	val	Y	Y	Y	N	seed	Y	N	N
PEAR (search) [173]	L1	A/B	A/B	–	P	P	N	prod.	P	Y	N

are coded no on other grounds: AgentX-Model reports the best AUC over rounds and experiments, and EvoSkillRec scores every candidate on test in every generation while stating that the scores are never used for selection.

F4: No loop reports spread over repeated sessions. None of the 16 offline-selecting loops repeats an evolution session and reports the spread, and seven have partial evidence. None of the ten online-selecting loops repeats a session for its headline result. Of the seven, RecHarness and VirtualMLE average three runs without

Table 7: Headline counts with their range when "not stated" cells are read either way, who coded them, and the agreement on the criterion they rest on. Coding caveats and alternative readings are in the notes below, §9 and Table 11.

Count (finding)	Value (range)	If the six uncoded loops were added[e]	Coded by	Pre-reconciliation agreement
Offline loops not stating selection data (F2)	8 of 16 (so 4–12 may select on the data they report)[a]	8–14 of 16–22	two authors[f] (first coder alone: 7 of 17)	C2 κ 0.90 (all 27: 0.90)
Offline loops stating test-once use (F3)	2 of 16 (2–10)	2–8 of 16–22	two authors[f] (first coder alone: 3 of 17)	C3 κ 0.70 (all 27: 0.79)
Offline loops reporting spread over repeated sessions (F4)	0 of 16 (7 partial)	0–6 of 16–22	two authors[f]	C4 κ 0.87 (all 27: 0.91)
Same-task validation-to-test checks (F5)	1 of 16 (5 partial; 14 if the 9 online confirmations are counted as partial, a post hoc rule)[b]	1–7 of 16–22	two authors[f]	C5 κ 0.25 [0.08, 0.59] (all 27: 0.30)
Equal-budget non-LLM null searches (F6)	3 of 16 (2 over the same space, 1 also at a total budget incl. LLM cost, R5)	3–9 of 16–22	two authors[f]	C6 κ 0.54 (all 27: 0.65)
Loops reporting an evaluation incident (F7)	4 of 27 (3 by the agent; 3 say what caught it)	4–10 of 33	two authors[f]	incident κ not interpretable[c]
Range of broad-metric online lifts (F8)	+0.052% to +4.86% (17 loops)	not bounded	one author	none
Online-selecting loops correcting across rounds (F9)	0 of 10 (1 partial)[d]	0–6 of 10–16	one author	LLM re-coding: 9 of 10 loops agree
Online-selecting loops confirming afresh (F9)	1 of 10 (4 partial)	1–7 of 10–16	one author	LLM re-coding: 6 of 10 loops agree; same totals, but only 2 of the 4 partial loops are the same
Online experiments reporting an interval or a named test for the headline (F9)	3 of 19 (7 counting bare claims of significance)	3–9 of 19–25	one author	LLM re-coding: 19 of 19 loops agree on the count of 7; the strict count is not re-coded

Agreement: pre-reconciliation per-criterion Cohen's κ between the two author coders over the 16 offline-selecting loops, our primary agreement statistic (in parentheses, all 27; other variants: Appendix B). It is likely inflated and is not the reliability of the released codes (note f).
[a] Of the other eight, four select on the data they report and four on validation; seven of the eight not stating confirm online. Under the alternative reading of Population Evolution (Appendix E): three and nine (range 3–12).
[b] Online confirmations (O in Table 6) test transfer, not overfitting to the selection data; the data file codes them partial under a rule added after both codings (Appendix B).
[c] The second coder could see all four incident codes (κ 0.85; 0.87); each incident is explicit in its paper.
[d] The LLM re-coding codes PILOT's within-round calibration as a correction (1 of 10); we code it partial (Appendix B).
[e] If the six uncoded in-scope loops (§6) all fell in the stratum, on either side of the criterion; column 2's not-stated ranges are not added. F8's range could widen either way.
[f] Not independent: the second coder had access to the first coding throughout, and the released codes follow the second coder's sheet in 236 of 243 cells: in effect one author's reading (§9).
Single-cell readings that would change a count: Appendix E.

a standard deviation.[6] Population Evolution reports one run per configuration, with repeated evaluations of the reference score and unreported preliminary runs whose "ordering was consistent" [137]. A-MLE re-runs a candidate when within-run variance is high, which the authors say "materially reduced" single-seed promotions [46]. AgentX-Model, EvoPilot and Amazon estimate the noise of single models without repeating a session.[7] Several papers state that their gains are within the resolution of a single run: EvoSkillRec notes margins "on the order of 10^{-4} AUC, which a single evolution run cannot reliably separate from run-to-run variation" [92]. AgentX-Model reports standard deviations of 0.0010–0.0022 AUC over repeated runs with a fixed seed while counting an experiment as above baseline if it exceeds it by more than 10^{-6} [194]. AutoLR, in its audited configuration, promoted candidates after a single run with noise calibration disabled, a risk its authors name the "KEEP ratchet": each promoted model becomes the baseline for all later comparisons [213]. Among the online-selecting loops, Google runs six independent sessions with bracketed ranges, but only for an ablation. For these loops a repeat is only practical on a replay (R4 in §7).

[6] RecHarness does so on its Amazon datasets; it is not stated for KuaiRec.
[7] EvoPilot notes that two publications of the same model, configuration and pinned data "differed by 0.36 points" [42], and Amazon re-evaluates one model across 11 dates ("6.18% ± 0.28 pp") [18].

F5 (descriptive): How loops check that selected gains transfer. Of the 16 offline-selecting loops, one checks its selected candidates on held-out data of the same task (EvoSkillRec), nine confirm a kept candidate online, five check on other data, scale, users or dates (Population Evolution, RankEvolve-HSTU, VirtualMLE, Self-EvolveRec, Amazon; Self-EvolveRec's held-out users are simulated, the weakest of the five) and one has none (A-MLE). EvoSkillRec's check is inconclusive: of five candidates on its validation Pareto front, only the one with the highest validation AUC keeps its gain on test, by a margin from a single run of the order F4 says a single run cannot separate from noise. All nine online confirmations report a gain, though only for the one candidate that reached traffic. The evidence of gains that do not survive comes from two sources. One is transfer across datasets and scale, which asks whether a gain generalises rather than whether the selection data were overfitted; the other is comparison points outside recommendation. In Population Evolution most locally selected ranking and RL candidates do not pass at larger scale [137], and in EvoRank the gains of the first campaign "were almost entirely fitness noise" [136]. None of this measures how much of the published recommender gains is selection noise; it shows that the question cannot be settled without held-out checks. Appendix C gives the numbers.

Table 8: Shortcuts reported in the audited loops (F7): what happened, what caught it where the paper says, and a mitigation that would remove or reduce it. RankEvolve-HSTU's diagnosis of negatives is a step the loop requires, not its acceptance rule. Two kinds of problem are not listed, though both loops are counted. One is reliability failures the paper does not report as raising a candidate's metric (EvoPilot's history rewriting that silently dropped an intended edit, its evaluator drift and a funnel mismatch that produced a false drop). The other is evaluation noise (RankEvolve-HSTU's subset evaluation, whose "phantom" 0.005-scale gains the paper reports chasing). The evidence files quote them.

Failure	Where	Caught by	Mitigation
Metric definition changed (batch size inflates an in-batch metric; in an exploratory session)	RecEvolve	a human	freeze metric-relevant settings
Target leakage in features or training pairs	RankEvolve-HSTU	the agent, from anomalous leaderboard trajectories	leakage checks before search (NOVA, EvoRank)
Evaluation protocol changed (multi-position scoring)	RankEvolve-HSTU	mandatory diagnosis of negatives	evaluator the agent cannot edit
Evaluation date inside the training window	EvoPilot	not stated (the result was retracted)	date-order checks
Proxy metric gamed (codebook size)	Amazon AutoResearch	human QA between runs	a second, usability metric
Capacity silently increased	AI Co-Scientist	not stated	matched-capacity comparisons

F6 (descriptive): Comparisons against a null search. Of the 16 offline-selecting loops, three compare against a non-LLM search at an equal or larger trial budget (in any search space; Table 2), five have partial controls and eight have none. The three are VirtualMLE (grid search and Bayesian optimisation), NOVA (Optuna-TPE) and EvoSkillRec (NASRec, alongside OpenEvolve), and in all three the LLM loop is reported to win. Two of the nulls (VirtualMLE's and NOVA's) search the loop's own space, and only VirtualMLE's also at a total budget that includes the LLM cost (R5). No loop runs an equal-budget random search over its own edit space, and the two random policies we found were not dominated. The control matters because random search with early stopping matched leading NAS methods [89] and classical optimisers beat LLM agents in fixed hyperparameter spaces [41, 148]. Appendix C describes each control.

F7 (provisional): Four loops report shortcuts in their own evaluation. Four of the 27 core loops and one comparison point report metric-winning bugs, metric inflation or leakage in the loop's own evaluation (Table 8). The agent introduced three of them (RecEvolve's in an exploratory session with the loop's own agent and evaluator); EvoPilot's is not attributed. None is reported as a headline gain. Where a paper says what caught a shortcut, it was a person (RecEvolve, Amazon) or the agent's diagnosis of anomalous results (RankEvolve-HSTU), not the keep-or-reject rule. That is expected rather than shown by three papers: a keep rule that scores the metric a shortcut inflates cannot reject it, so the check must sit outside the metric. Four incidents in 27 loops say little about how often shortcuts go unreported, since preventive guards (EvoRank's pre-search leakage audit [136], NOVA's verifier [109]) leave nothing to report. These papers should be credited for reporting (R8). The second coder also flagged AutoLR (Appendix B). Appendix C gives each incident and the isolation each loop states.

F8 (descriptive): How baselines, budgets and openness are reported. Some large relative gains are measured against weak starting points, few loops describe tuning their baselines, and budgets are reported in incompatible units. The 17 loops that report an online lift on a broad metric (one a whole surface is judged by) report from +0.052% to +4.86%. This counts a fresh confirmation where the paper reports one and otherwise the reported figure; taking Sortify's selecting round instead raises the upper end to +9.2%. The range is descriptive, since metrics, designs and baselines differ, and it is the same without PILOT's non-randomised before/after lift and A/B Agent's selecting-round figure (Appendix E). Only VirtualMLE and Amazon's AutoResearch describe tuning their baselines, and only VirtualMLE says on which data (a third, Population Evolution, calibrates its baseline on the test split, so the lenient R7 proxy in Table 10 counts three). Twelve of 27 report LLM tokens or cost, four release code, two promise to, and six do not name the LLM they use. Appendix C gives the examples and the counting rules.

F9 (provisional): Online loops rarely correct across rounds or confirm afresh. Nineteen of the 27 loops report online experiments. Of the ten that select online, none corrects for multiple testing across rounds (one partially) and one confirms its chosen candidate afresh (four partially). The partial codes are unstable: a blind LLM re-coding (same model family as the extractor) gives the same totals but agrees on only two of the four partial loops. Online experiments are the main safeguard of the industrial loops and strong evidence when well designed. In nine loops that select offline, the A/B test checks one candidate chosen beforehand, so it is a confirmation by construction. In the ten online-selecting loops the experiment is the keep-or-reject rule, and there the headline effect is usually the selected round, although the arm chosen as best overstates its effect [11, 85]. Only Sortify re-tests its chosen round in a fresh experiment [30]. Across all 19, eight state the experiment duration and seven claim significance or report confidence intervals for the headline result, but only three of them report an interval or a named test. Appendix C gives the partial cases and the counting rules.

Practices worth copying. The audit also found reported practices that, if combined, would meet most of the checklist in §7; we did not test their effect. They are: RecHarness's validation-only selection with a single test evaluation; VirtualMLE's hidden test set, immutable pipeline, and grid-search and Bayesian-optimisation baselines; EvoSkillRec's validation-versus-test check of its Pareto front; Population Evolution's withheld transfer targets; NOVA's leakage verifier and EvoRank's pre-search leakage audit with an equal-budget random search; RankEvolve-HSTU's mandatory diagnosis of negative results; and Sortify's fresh re-test and PILOT's pre-registered confirmation design.

Does selection really overfit? A meta-analysis of 120 Kaggle competitions found little overfitting to public leaderboards [149], and

new test sets for ImageNet showed accuracy drops but preserved model rankings [144], partly because similar models overfit a reused test set less than theory predicts [117]. Agent-proposed candidates are near-copies of an incumbent, which by that argument limits regressional overfitting. Two differences remain: an agent loop evaluates hundreds of correlated candidates on one split without a resubmission cap [13] and keeps the maximum, and it controls the code that produces the score, which no Kaggle team does. The direct measurements (F5) show that some selected gains shrink or vanish on transfer across datasets and scale while others survive; the one same-task check settles nothing. The safe conclusion is not that self-evolving recommenders do not work. It is that, for gains selected and reported offline, the published evidence cannot yet tell how much is real. An online confirmation is direct evidence for the one candidate that reached traffic. For the online-selecting loops, whose A/B gate is direct evidence, the open question is the selection across rounds (F9).

7 A Proposed Reporting Checklist

We argue, rather than show, that most of the problems are cheap to report: what the audit finds missing is mostly a statement of what was done, not a new experiment (Appendix A applies the minimum tier to two papers). Table 9 turns the codebook into a proposed reporting checklist. Each item has a minimum that any paper can meet by reporting what it already did, and a recommended practice that costs compute. Each maps to the audit criterion that checks it, so that a paper's compliance can be read off the same coding. The items extend general checklists for NAS research [101], ML-based science [73] and agent benchmarks [230] to the specific failure points of an evolution loop (Figure 1). Several items are established practice restated for loops: R1 follows the recommender-evaluation literature on sampled metrics [82], R4 the reporting of seeds and variance in deep RL and NLP [1, 35, 56], R5 the random-search baseline of the NAS checklist [101], R6 cost-aware agent evaluation [75], and R9 the practice of trustworthy online experiments [80]. What is specific to self-evolving systems is that each item applies to a *chain* of selection stages and to an optimiser that can change its own measurement. R2 asks for the selection signal at every stage, not for one split. R4 counts evolution sessions, not training seeds. R5 asks for a null over the loop's own edit space, and R8 asks what the agent can read and write. R9 does not ask for a correction across a platform's independent launches, which are usually governed one launch at a time with guardrails or false-discovery control. It addresses the case where one loop picks the best of its own rounds and reports that round's effect, which the winner's curse biases upward [85]; the remedy is a correction across those rounds or, more simply, a fresh confirmation whose effect is the one reported. Appendix A applies the minimum tier to two audited papers (a core loop and a learning-to-rank comparison point). Table 10 reads the audit coding as a retrospective proxy for the checklist; this is a check by the authors on codes recorded for another purpose, not an application of the checklist by independent raters, and its R1, R2, R6 and R8 proxies are lenient. *We therefore estimate no rate of compliance with the minimum tier.* Most papers state some budget (R6); the R2 and R8 proxies are met mostly by construction (the online gate; bounded actions). Few say how their baselines were tuned (R7) or report both duration and an interval or significance test for their online result (R9). For R3, R4 and R5 the coding records only the recommended practice, which almost no paper follows. Stricter readings of each item are in Appendix A.

Cost of compliance. The minimum tier only asks authors to report what they did, so it costs little beyond writing. Industrial authors may be unable to disclose some items, such as traffic shares or the exact data split; they can then say that an item is withheld rather than leave it unstated. The minimum tier does not make a weak design acceptable, but it makes it visible. In the recommended tier, R4 and R5 multiply training cost: three sessions plus one equal-budget null cost four sessions. For an academic harness such as RecHarness, whose budget is 43,200 GPU-seconds per backbone [102], that is 48 GPU-hours per backbone. LLM costs are small next to training: VirtualMLE reports $1.5–4.8 of LLM cost per session [16], EvoRank about ten dollars for fifty iterations [136], and NOVA about $29 per task, or about $22 with cached artifacts [109] (LLM cost only). For industrial loops whose candidates take hundreds of GPU-hours [90], R4 and R5 can be applied to a replay or a reduced-scale proxy, as AgentX-Model does for scheduling policies [194], while R9 governs the launch decision.

A shared protocol. Several items can be enforced in software rather than requested in prose: a sealed test split (R2, R3), a single final evaluation, an equal-budget null evolver (R5), a budget card (R6) and an evaluator outside the agent's reach (R8). Appendix F describes recforge [143], a reference implementation that we are building. It enforces R2, R3 and R5 through its API but not yet R8, and its seal holds only against code that uses the API. Its current protocol is leave-one-out, and we report no results from it.

8 Open Problems

Isolation without crippling the agent. Code-level agents need to read data and change training code, and that is exactly the access that lets them leak targets or change what the metric measures (F7). Verification gates (NOVA), fixed evaluation code (AgentX-Model), immutable pipelines (VirtualMLE) and cross-agent review (RankEvolve-HSTU) each close part of the gap. A principled design would give the agent a process that physically cannot reach evaluation labels or code, and would audit every metric-relevant change, such as batch size for in-batch metrics.

Proxies for expensive candidates. Industrial candidates take hours to hundreds of GPU-hours, so loops screen on proxies: small decoders [129], data subsets and peer datasets [137], training-loss curves and learned reward models [60], and LLMs can themselves predict a candidate architecture's performance before training [66]. The evidence shows that proxy fidelity depends on what is evolved. Characterising when a proxy can be trusted, and spending budget to re-check proxies during search, is open.

Budget-normalised comparison of evolvers. There is no common unit of cost and no common task, so the field cannot say whether bandit routing, island evolution, research-agent pairs or learned proposers are better. A shared benchmark with a budget card (R6) and an equal-budget random baseline (R5) would allow the first such comparison.

Table 9: Proposed reporting checklist for self-evolving recommenders. *Minimum*: report what was done; *recommended*: what a strong study does. The last column gives the audit criterion (Table 2) or finding that checks the item.

	Minimum (report)	Recommended (do)	Audit
R1 Metric	State the candidate set and how negatives are drawn (full catalogue, N sampled, in-batch); for ranking-stage models, the traffic on which AUC or logloss is computed.	Full-catalogue ranking where the catalogue allows, sampled metrics only as secondary [82].	C1
R2 Selection	Name the data or signal that accepts candidates at every stage of the chain (proxy, offline, online).	Keep the test split out of every decision, including baseline tuning.	C2
R3 Test once	Report the selected candidate's validation score next to its test score.	Evaluate on test once, after the search.	C3, C5
R4 Sessions	State how many independent sessions were run (one is acceptable if stated) and report every session's final test score.	≥ 3 sessions, ideally ≥ 5, with an interquartile mean and bootstrap interval [1], and the expected best score as a function of budget [35]. Industrial loops: repeat on a replay or a reduced-scale proxy.	C4
R5 Null control	State which null controls were run and at what trial and compute budget, or that none was.	At least one non-LLM null over the loop's own edit space at equal trial and compute budget (the loop's LLM cost counted as compute): for a configuration-level loop, HPO or random search over the knobs that the kept edits changed; for a code-level loop, random mutation over its edit arms or grammar, since HPO over the changed knobs is necessary but not sufficient there (§5.2). Optionally, the same proposer without evaluation feedback or with candidates accepted at random.	C6
R6 Budget	A budget card: trials, GPU-hours, LLM tokens and cost, wall-clock, human hours, and the cost of training a learned proposer.	Report results as a function of budget, not only at its end [75].	C7
R7 Baselines	How the starting point and compared models were tuned, and on which data.	Tune them with the budget of one session.	C8
R8 Access	What the agent can read and write (data, labels, evaluation code, metric definitions); every gaming or leakage incident and what caught it.	An evaluator in a process the agent cannot modify; review of metric-relevant changes (batch size, candidate set, evaluation date).	C9
R9 Online gates	Duration, traffic share, pre-specified primary metric and guardrails, confidence intervals, and the number of rounds tested.	A correction across the loop's rounds (e.g. false-discovery control) or a fresh, pre-registered confirmation of the final candidate, reporting the confirmed rather than the selected effect [80].	F9

Evolving generative recommenders. Semantic-ID construction, codebook collisions, constrained decoding and preference-alignment losses are new design dimensions with little human prior and large effects. They are natural targets for code-level evolution, and almost untouched (§5).

From offline wins to online value. RecEvolve observes that offline evaluations often show "win-win" results while online tests show trade-offs [135]. Loops that select on offline metrics inherit the gap, and loops that select on online experiments face multiple testing and slow iteration. A starting point is to combine the two with a fresh online confirmation of the final candidate, ideally pre-registered, as Sortify does with a seven-day re-test.

Circularity at the policy level. L4 systems are often judged by simulators from the same model family as the agent being improved. Held-out simulators [78], verifiable rewards [125] and reward-model benchmarks [204] are partial answers.

9 Limitations

This section collects the limits of the audit in one place; Table 11 summarises who coded each part and how it was checked.

Coverage. Our cut-off is 22 October 2026, and we coded the arXiv version available on that date. Generic keyword search performed poorly, so coverage relies on citation-graph search, with titles screened by one screener, and may miss systems that neither cite nor are cited by the core set. A recall check within that network, run with an LLM agent on 27 October, found 22 of 24 cited candidate systems named in the survey, but it is not independent of the search. The first sweep missed five of the 27 core loops and four more were recognised only after coding, so the corpus is not saturated. 32 of the 34 loops we found are from 2026, but our arXiv screen began in 2025, so the field may be older than the corpus suggests (Appendix B). Six known in-scope systems are not coded; adding them could move any core count by at most six (Table 7).

Coding. Two authors coded the scored criteria and the incident flag, not independently, so the evidential weight is closer to one careful coder with a consistency check than to double coding. The first coded from verbatim quotes extracted by LLM agents and checked against the paper text. The second coded the full texts by hand, without an LLM by the coder's own account, from a guide written by an author who knew the corpus. The first coding's files were in the shared repository throughout, kept from the second coder only by the guide's instruction, and 17 of its cells, among them

Table 10: Retrospective proxy mapping of the proposed reporting checklist on the 27 core papers, read off the audit coding by the authors; it is not an application of the checklist by independent raters, and the last column is not a compliance rate: most proxies are lenient (strict readings: R1 10 of 15, without RecEvolve, whose candidate set is stated only in its incident description; R5 1 of 27, R9 2 of 19; R2 is met by 8 of the 16 offline-selecting loops). Each item is approximated by the audit code shown; rows marked rec. proxy the recommended tier only, and the minimum tier of R3–R5 is not coded, so no row measures minimum-tier compliance. The notes below give each item's tier, leniency and denominator.

Item	Tier	Proxy	Lenient proxy satisfied (set or this loop)
R1 Metric	min.	candidate set or AUC stated	11/15
R2 Selection	min.	selection data stated	18/26
R3 Test once	rec.	test used once (C3)	2/16
R4 Sessions	rec.	repeats with spread (C4)	0/27
R5 Null control	rec.	any equal-budget non-LLM search (C6)	3/27
R5 Null control	rec.	same space, total budget incl. LLM cost (F6)	1/27
R6 Budget	min.	some budget stated (C7)	24/27
R7 Baselines	min.	baseline tuning described (C8)	3/27
R8 Access	min.	access design stated (C9)	24/27
R9 Online gates	min.	duration and CI or sig. stated	4/19
R9 Online gates	rec.	correction and confirmation	0/10

min.: the code checks part of the minimum tier, so the count is an upper bound on papers meeting the minimum (except R7, Appendix A). *rec.*: the minimum (for R3, R4 and R5, stating what was done) is not coded, so the count is not a bound on the minimum: it counts papers that already follow the recommended practice. The R1, R2, R6 and R8 proxies are lenient, R3's reads C3 only, although Table 9 also maps R3 to C5, and R5's counts any equal-budget non-LLM search. Denominators: R1, core loops whose headline is a ranking metric or AUC; R2, all except the operational loop; R3, the 16 offline-selecting loops (including AutoLR, which selects offline but headlines only online lifts); R9, loops with an online experiment (min.) or that select online (rec.); otherwise all 27. Appendix A details each proxy's leniency.

Table 11: Who coded what, how it was checked, and which findings rest on it. All human coders are authors. Details: the Coding paragraph below and Appendix B.

Material	Coded by	Check	Findings resting on it
Scored criteria and incident flag, 27 core loops	two authors, not independently (second coder had repository access); released codes follow the second in 236/243 cells	reconciliation; κ 0.68 offline (C5 0.25, C6 0.54, C8 0.56); likely inflated	F1–F8, strata
Online-gate items, 19 loops	one author	blind LLM re-coding; 88/95 cells agree before adjudication (κ 0.86); three changed afterwards	F9
Seven comparison points	one author	LLM checks, author-reviewed	contrasts in F5–F7
Design fields, kept edits, F8 lift count	one author	no second coding	§3.1, §5.2, F8, Table 3
Title screening	one screener, LLM-assisted	citation check, 22 of 71	corpus, recall unmeasured

all four incident codes, were visible in the codebook file, so agreement before reconciliation is likely inflated. Criterion-stratified κ (chance agreement taken within each criterion) is 0.68 (95% interval 0.55–0.77) over the 16 offline-selecting loops and 0.71 (0.60–0.79) over all 243 cells; C5 (0.25), C6 (0.54) and C8 (0.56) are the weakest. Of the 38 disagreements, 31 were resolved to the second coding under rules written during reconciliation, so κ is not the reliability of the released codes. F9's online items were coded by one author on 27 October, after the cut-off, and checked by a blind re-coding with an LLM of the same family as the extractor. Appendix B gives the alternative statistics, batch timing, guide changes and the cells that rest on reconciliation; every change to a cell is in the released changelog.

What the counts can and cannot say. "Not stated" does not mean "not done": authors may have followed a practice without reporting it, and a loop coded "not stated" may well have been evaluated soundly. The findings describe what the coded arXiv versions report, not the quality of the deployed systems. Offline hygiene criteria mean little for loops whose keep-or-reject decision is an online experiment, which is why those loops are assessed on the design of their experiments (F9). The audited authors have not yet verified their rows. We will maintain the audit as a living table: new systems will be coded with the same codebook and second-coded, each update will be a versioned, archived release with a cell-level changelog, and corrections from authors will be accepted with a quote from their paper and checked by a second coder. A correction to a cell coded not stated moves a count only within its stated range (F2 between 4 and 12, F3 between 2 and 10); a correction to a cell coded yes or no can move it outside.

Conflict of interest. The authors are building recforge, the reference implementation described in Appendix F. The audit does not depend on it, but recforge enforces R2, R3 and R5 of the checklist, so we have an interest in the checklist's adoption. It has produced no evolver results, and its isolation is API-level only. One of us wrote the ideation-harness study cited in §5, which is outside recommendation and not peer reviewed, and both of us wrote the LLM-guided regularisation paper cited in §2 as an excluded example.

10 Conclusion

In the papers we found, nearly all from 2026, self-evolving recommenders range from research prototypes to loops that their authors report in production at many large platforms. The loops are inventive, and many report online gains from production A/B tests. Their offline evidence is sparsely reported: of 16 offline-selecting loops, 8 do not state their selection data, 2 state test-once use, and none reports spread over repeated sessions. Null searches at an equal or larger trial budget are rare, and whether selected gains transfer cannot be told from the corpus. Online gates are reported but seldom corrected for the rounds they test or confirmed afresh. Four loops report shortcuts in their own evaluation; where a paper says what caught one, it was a person or the agent's own diagnosis, which is what to expect when the keep rule scores the inflated metric. Guards outside that metric are therefore needed. These are counts of what the papers report, coded by the authors (§6), and an outside re-coding is the next check. We expect, but have not shown, that most of what is missing is cheap to report. We hope that the checklist in Table 9, the released audit and design data, and protocols that enforce the checklist in software make it the norm, so that the next comparison of evolvers can be made on equal terms.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. 2021. Deep Reinforcement Learning at the Edge of the Statistical Precipice. In *Advances in Neural Information Processing Systems*, Vol. 34. Curran Associates, Inc., 29304–29320. arXiv:2108.13264 https://proceedings.neurips.cc/paper_files/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html

[2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2623–2631. doi:10.1145/3292500.3330701

[3] Vito Walter Anelli, Alejandro Bellogín, Antonio Ferrara, Daniele Malitesta, Felice Antonio Merra, Claudio Pomo, Francesco Maria Donini, and Tommaso Di Noia. 2021. Elliot: a Comprehensive and Rigorous Framework for Reproducible Recommender Systems Evaluation. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 2405–2414. arXiv:2103.02590 doi:10.1145/3404835.3463245

[4] Dongyang Ao, Kaixiang Fang, and Shijie Xu. 2026. RecSys Factory: Bounding LLM Agent Autonomy to Decision Points in the Industrial Recommender Lifecycle. arXiv:2608.11241. arXiv:2608.11241 https://arxiv.org/abs/2608.11241

[5] Muhammad Rafay Azhar, Yuhang Zhou, Gilbert Jiang, Yuchen Wang, Rahul Sharma, Matthew DeSousa, Jiayi Liu, Xin Guo, Lizhu Zhang, and Xiangjun Fan. 2026. CORAL: An LLM-Native Harness for Production Recommender Systems. arXiv:2609.02730. arXiv:2609.02730 https://arxiv.org/abs/2609.02730

[6] Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. 2025. Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv:2503.11926. arXiv:2503.11926 https://arxiv.org/abs/2503.11926

[7] Merwan Barlier and Blaž Škrlj. 2026. LLMs as Feature Engineers for Text-and-Tabular Prediction. arXiv:2609.21894. arXiv:2609.21894 https://arxiv.org/abs/2609.21894

[8] Moritz Baumgart, Philipp Meister, Justus Krell, Michael Schmidt, Bela Gipp, and Joeran Beel. 2026. AutoRecLab: An Autonomous Recommender Systems Lab. In *Proceedings of the 20th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 1790–1792. arXiv:2609.21863 doi:10.1145/3773078.3841273

[9] Joeran Beel, Bela Gipp, Tobias Vente, Moritz Baumgart, and Philipp Meister. 2025. From AutoRecSys to AutoRecLab: A Call to Build, Evaluate, and Govern Autonomous Recommender-Systems Research Labs. arXiv:2510.18104. arXiv:2510.18104 https://arxiv.org/abs/2510.18104

[10] Joeran Beel, Min-Yen Kan, and Moritz Baumgart. 2025. Evaluating Sakana's AI Scientist: Bold Claims, Mixed Results, and a Promising Future? arXiv:2502.14297. arXiv:2502.14297 https://arxiv.org/abs/2502.14297

[11] Ron Berman, Walter W. Zhang, and Hangcheng Zhao. 2026. Valuing Winners: When and How to Correct for Selection Bias in Randomized Experiments. arXiv:2605.18887. arXiv:2605.18887 https://arxiv.org/abs/2605.18887

[12] Martin Andres Bertran, Aaron Roth, and Zhiwei Steven Wu. 2026. What Fits (Into Few Tokens) Doesn't Overfit: Compression and Generalization in ML Research Agents. arXiv:2606.11045. arXiv:2606.11045 https://arxiv.org/abs/2606.11045

[13] Avrim Blum and Moritz Hardt. 2015. The Ladder: A Reliable Leaderboard for Machine Learning Competitions. In *Proceedings of the 32nd International Conference on Machine Learning (ICML), PMLR 37*. 1006–1014. arXiv:1502.04585 https://arxiv.org/abs/1502.04585

[14] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. 2021. Accounting for Variance in Machine Learning Benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 3. 747–769. arXiv:2103.03098 https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html

[15] Shihao Cai, Jizhi Zhang, Keqin Bao, Chongming Gao, Qifan Wang, Fuli Feng, and Xiangnan He. 2024. Agentic Feedback Loop Modeling Improves Recommendation and User Simulation. arXiv:2410.20027. arXiv:2410.20027 https://arxiv.org/abs/2410.20027

[16] Shiteng Cao, Jingwen Liu, Junda She, and Zhiheng Li. 2026. VirtualMLE: A Virtual ML Engineer that Optimizes Sequential Recommenders. arXiv:2606.03221. arXiv:2606.03221 https://arxiv.org/abs/2606.03221

[17] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. 2025. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.07095 https://arxiv.org/abs/2410.07095

[18] Aparajith Chandran, Juwon Kim, Saurav Jha, Pablo Castells, and Florian Hottier. 2026. AutoResearch at Production Scale: Failure Modes and a Multi-Agent Framework. arXiv:2609.30541. arXiv:2609.30541 https://arxiv.org/abs/2609.30541

[19] Olivier Chapelle, Thorsten Joachims, Filip Radlinski, and Yisong Yue. 2012. Large-Scale Validation and Analysis of Interleaved Search Evaluation. *ACM Transactions on Information Systems* 30, 1, Article 6 (2012). doi:10.1145/2094072.2094078

[20] Angelica Chen, David M. Dohan, and David R. So. 2023. EvoPrompting: Language Models for Code-Level Neural Architecture Search. In *Advances in Neural Information Processing Systems 36*. arXiv:2302.14838 https://arxiv.org/abs/2302.14838

[21] Bo Chen, Xiangyu Zhao, Yejing Wang, Wenqi Fan, Huifeng Guo, and Ruiming Tang. 2022. A Comprehensive Survey on Automated Machine Learning for Recommendations. arXiv:2204.01390. arXiv:2204.01390 https://arxiv.org/abs/2204.01390

[22] Hanchong Chen, Xing Tang, Lingjie Li, Xiongfeng Shan, and Xiuqiang He. 2026. When Memory Takes Gradients: Collaborative Vector Memory for Agentic Recommender Systems. arXiv:2608.26895. arXiv:2608.26895 https://arxiv.org/abs/2608.26895

[23] Hui Chen, Miao Xiong, Yujie Lu, Wei Han, Ailin Deng, Yufei He, Jiaying Wu, Yibo Li, Yue Liu, and Bryan Hooi. 2025. MLR-Bench: Evaluating AI Agents on Open-Ended Machine Learning Research. arXiv:2505.19955. arXiv:2505.19955 https://arxiv.org/abs/2505.19955

[24] Runjin Chen, Mingxuan Ju, Ngoc Bui, Dimosthenis Antypas, Stanley Cai, Xiaopeng Wu, Leonardo Neves, Zhangyang Wang, Neil Shah, and Tong Zhao. 2024. Enhancing Item Tokenization for Generative Recommendation through Self-Improvement. arXiv:2412.17171. arXiv:2412.17171 https://arxiv.org/abs/2412.17171

[25] Weixin Chen, Yuhan Zhao, Jingyuan Huang, Zihe Ye, Clark Mingxuan Ju, Tong Zhao, Neil Shah, Li Chen, and Yongfeng Zhang. 2026. MemRec: Collaborative Memory-Augmented Agentic Recommender System. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 44515–44544. arXiv:2601.08816 doi:10.18653/v1/2026.acl-long.2061

[26] Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. 2025. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. In *International Conference on Learning Representations*. arXiv:2410.05080 https://arxiv.org/abs/2410.05080

[27] Ziqi Chen and Mohamed Hammad. 2026. Open-Ended Autonomous Model Research for Recommendation Systems. In *NeurIPS 2026 Workshop on Autonomous Machine Learning Research*. https://openreview.net/forum?id=91mjHN3kDp

[28] Zheng Chen, Linfeng Liu, Hong Li, and Hong Yan. 2026. RankEvolve: A Reliable Multi-Agent Auto-Research Harness for Evolving Ranking Models. arXiv:2609.39551. arXiv:2609.39551 https://arxiv.org/abs/2609.39551

[29] Zhi Chen, Minmao Wang, Xingchen Liu, Haoqiang Liang, Huihuang Lin, Likang Wu, Hongke Zhao, Yulong Wang, Shijie Yi, Fei Pan, and Peng Jiang. 2026. From Understanding to Action: Feedback-Grounded Policy Discovery for Generative Recommendation. arXiv:2607.27789. arXiv:2607.27789 https://arxiv.org/abs/2607.27789

[30] Yin Cheng, Liao Zhou, Xiyu Liang, Dihao Luo, Tewei Lee, Kailun Zheng, Weiwei Zhang, Mingchen Cai, Jian Dong, and Andy Zhang. 2026. Let the Agent Steer: Closed-Loop Ranking Optimization via Influence Exchange. arXiv:2603.27765. arXiv:2603.27765 https://arxiv.org/abs/2603.27765

[31] Maurizio Ferrari Dacrema, Simone Boglio, Paolo Cremonesi, and Dietmar Jannach. 2021. A Troubling Analysis of Reproducibility and Progress in Recommender Systems Research. *ACM Transactions on Information Systems* 39, 2, Article 20 (2021), 49 pages. arXiv:1911.07698 doi:10.1145/3434185

[32] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. 2019. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 101–109. arXiv:1907.06902 doi:10.1145/3298689.3347058

[33] Alex Deng, Ya Xu, Ron Kohavi, and Toby Walker. 2013. Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data. In *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining*. ACM, New York, NY, USA, 123–132. doi:10.1145/2433396.2433413

[34] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. arXiv:2502.18965. arXiv:2502.18965 https://arxiv.org/abs/2502.18965

[35] Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. 2019. Show Your Work: Improved Reporting of Experimental Results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2185–2194. arXiv:1909.03004 doi:10.18653/v1/D19-1224

[36] Xuanyi Dong and Yi Yang. 2020. NAS-Bench-201: Extending the Scope of Reproducible Neural Architecture Search. In *International Conference on Learning*

Representations (ICLR). arXiv:2001.00326 https://openreview.net/forum?id=HJxyZkBKDr

[37] Miroslav Dudik, John Langford, and Lihong Li. 2011. Doubly Robust Policy Evaluation and Learning. In *Proceedings of the 28th International Conference on Machine Learning*. arXiv:1103.4601 https://arxiv.org/abs/1103.4601

[38] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. 2015. Generalization in Adaptive Data Analysis and Holdout Reuse. In *Advances in Neural Information Processing Systems 28*. arXiv:1506.02629 https://arxiv.org/abs/1506.02629

[39] Katharina Eggensperger, Philipp Müller, Neeratyoy Mallik, Matthias Feurer, René Sass, Aaron Klein, Noor Awad, Marius Lindauer, and Frank Hutter. 2021. HPOBench: A Collection of Reproducible Multi-Fidelity Benchmark Problems for HPO. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, Vol. 1. arXiv:2109.06716 https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/93db85ed909c13838ff95ccfa94cebd9-Abstract-round2.html

[40] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. 2025. A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems. arXiv:2508.07407. arXiv:2508.07407 https://arxiv.org/abs/2508.07407

[41] Fabio Ferreira, Lucca Wobbe, Arjun Krishnakumar, Frank Hutter, and Arber Zela. 2026. Can LLMs Beat Classical Hyperparameter Optimization Algorithms? A Study on autoresearch. arXiv:2603.24647. arXiv:2603.24647 https://arxiv.org/abs/2603.24647

[42] Hao Fu, Baiting Zhu, Minglei Chen, Yinjie Huang, and Shuai Ding. 2026. Verify, Don't Trust: Agentic Model Development for Video Discovery Retrieval at Scale. arXiv:2609.21257. arXiv:2609.21257 https://arxiv.org/abs/2609.21257

[43] Chongming Gao, Ruijun Chen, Shuai Yuan, Kexin Huang, Yuanqing Yu, and Xiangnan He. 2025. SPRec: Self-Play to Debias LLM-based Recommendation. In *Proceedings of the ACM on Web Conference 2025*. ACM, New York, NY, USA, 5075–5084. arXiv:2412.09243 doi:10.1145/3696410.3714524

[44] Chongming Gao, Shijun Li, Wenqiang Lei, Jiawei Chen, Biao Li, Peng Jiang, Xiangnan He, Jiaxin Mao, and Tat-Seng Chua. 2022. KuaiRec: A Fully-observed Dataset and Insights for Evaluating Recommender Systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. ACM, 540–550. doi:10.1145/3511808.3557220

[45] Chongming Gao, Shijun Li, Yuan Zhang, Jiawei Chen, Biao Li, Wenqiang Lei, Peng Jiang, and Xiangnan He. 2022. KuaiRand: An Unbiased Sequential Recommendation Dataset with Randomly Exposed Videos. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. ACM, 3953–3957. doi:10.1145/3511808.3557624

[46] Erwin Gao, Vinodh Kumar Sunkara, Jingyi Guan, Qinjin Jia, Hangjun Xu, Xiang Ji, Sherman Wong, Surya Teja Chavali, Pratik Vaishnavi, Aryan Pandhi, Xiaoyu Deng, Zhaodong Wang, Samarth Inani, Fan Yang, Jakob Moberg, Zoe Zu, Nicolas Bievre, Sami Khenissi, Amit Jaspal, Ehsan Fakharizadi, Srinidhi Viswanathan, Dorothy Sun, Abishek Vanam, Sneha Iyer, Sheela Yadawad, Wenjie Chen, Gaby Nahum, Junhua Gu, Peter Chu, Yucheng Liu, Xin Zhao, Vitor Cid, Chaorong Chen, Vijay Pappu, Ashwin Kumar, Wenlin Chen, Ben Schulte, Deepak Chandra, and Ritwik Tewari. 2026. Agentic ML Exploration (A-MLE) for Ads Ranking. arXiv:2609.08248. arXiv:2609.08248 https://arxiv.org/abs/2609.08248

[47] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. 2026. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. *Transactions on Machine Learning Research* (2026). arXiv:2507.21046 https://arxiv.org/abs/2507.21046

[48] Leo Gao, John Schulman, and Jacob Hilton. 2023. Scaling Laws for Reward Model Overoptimization. In *Proceedings of the 40th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 10835–10866. arXiv:2210.10760 https://proceedings.mlr.press/v202/gao23h.html

[49] Pratham Grover. 2026. Separating Real Improvements from Lucky Runs in Autonomous ML Research. In *NeurIPS 2026 Workshop on Autonomous Machine Learning Research*. https://openreview.net/forum?id=x3lY2QO0Xe

[50] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. EvoPrompt: Connecting LLMs with Evolutionary Algorithms Yields Powerful Prompt Optimizers. In *International Conference on Learning Representations*. arXiv:2309.08532 https://arxiv.org/abs/2309.08532

[51] Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. 2024. DS-Agent: Automated Data Science by Empowering Large Language Models with Case-Based Reasoning. arXiv:2402.17453. arXiv:2402.17453 https://arxiv.org/abs/2402.17453

[52] Ankai Hao, Ke Chen, Huan Li, and Lidan Shou. 2026. LATTEArena: An Evaluation Framework for LLM-powered Tabular Feature Engineering (Extended Version). arXiv:2606.09004. arXiv:2606.09004 https://arxiv.org/abs/2606.09004

[53] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4, Article 19 (2015), 19 pages. doi:10.1145/2827872

[54] Tonmoy Hasan, Taylor Foust, Shao Tang, Leonardo Neves, Aman Gupta, Hiroto Udagawa, Helder Dias, Daniel Silva, and Rohan Ramanath. 2026. From Valid to Useful: Post-Verification Acquisition for Recursive Self-Improving Recommendation. arXiv:2610.04302. arXiv:2610.04302 https://arxiv.org/abs/2610.04302

[55] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *Proceedings of the 26th International Conference on World Wide Web*. International World Wide Web Conferences Steering Committee, 173–182. doi:10.1145/3038912.3052569

[56] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. Deep Reinforcement Learning That Matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32. AAAI Press. arXiv:1709.06560 doi:10.1609/aaai.v32i1.11694

[57] Balázs Hidasi and Ádám Tibor Czapp. 2023. Widespread Flaws in Offline Evaluation of Recommender Systems. In *Proceedings of the 17th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 848–855. arXiv:2307.14951 doi:10.1145/3604915.3608839

[58] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *International Conference on Learning Representations (ICLR)*. arXiv:1511.06939

[59] Jinxin Hu, Hao Deng, Lingyu Mu, Hao Zhang, Shizhun Wang, Yu Zhang, and Xiaoyi Zeng. 2026. Rethinking Recommendation Paradigms: From Pipelines to Agentic Recommender Systems. arXiv:2603.26100. arXiv:2603.26100 https://arxiv.org/abs/2603.26100

[60] Jinxin Hu, Hao Deng, Haibo Xing, Lingyu Mu, Muyu Zou, Weiqin Yang, Sirui Chen, Bohao Wang, Zhezheng Hao, Hao Zhang, Zulong Chen, Shizhun Wang, Yu Zhang, Xiaoyi Zeng, and Jiawei Chen. 2026. Astar: Learning to Propose Evolution Directions for Self-Evolving Industrial AI Systems. arXiv:2608.27287. arXiv:2608.27287 https://arxiv.org/abs/2608.27287

[61] Litao Hu and Yutong Tang. 2026. The Winner's Curse in LLM Self-Improvement Loops: Selection Noise, Lock-in, and Acceptance Rules. arXiv:2610.09239. arXiv:2610.09239 https://arxiv.org/abs/2610.09239

[62] Peiyu Hu, Weihai Lu, Siying Gu, Zhuodong Liu, Zhaokai Luo, Yuean Niu, Zhiyong Wang, and Jia Wang. 2026. AtomRec: Evolving Atomic Memory for Agentic Recommendation. arXiv:2609.04882. arXiv:2609.04882 https://arxiv.org/abs/2609.04882

[63] Chengkai Huang, Hongtao Huang, Tong Yu, Kaige Xie, Junda Wu, Shuai Zhang, Julian Mcauley, Dietmar Jannach, and Lina Yao. 2025. A Survey of Foundation Model-Powered Recommender Systems: From Feature-Based, Generative to Agentic Paradigms. arXiv:2504.16420. arXiv:2504.16420 https://arxiv.org/abs/2504.16420

[64] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. 2024. MLAgentBench: Evaluating Language Agents on Machine Learning Experimentation. In *Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR 235*. 20271–20309. arXiv:2310.03302 https://arxiv.org/abs/2310.03302

[65] Takashi Ishida, Taro Makino, and Issa Sugiura. 2026. QD-Rec: Open-Ended Evolution of Recommendations via Quality-Diversity. arXiv:2610.23470. arXiv:2610.23470 https://arxiv.org/abs/2610.23470

[66] Ganesh Jawahar, Muhammad Abdul-Mageed, Laks V. S. Lakshmanan, and Dujian Ding. 2023. LLM Performance Predictors are good initializers for Architecture Search. arXiv:2310.16712. arXiv:2310.16712 https://arxiv.org/abs/2310.16712

[67] Yitong Ji, Aixin Sun, Jie Zhang, and Chenliang Li. 2023. A Critical Study on Data Leakage in Recommender System Offline Evaluation. *ACM Transactions on Information Systems* 41, 3 (2023), 1–27. arXiv:2010.11060 doi:10.1145/3569930

[68] Zhuohang Jiang, Yuxin Chen, Yongsen Pan, Zheng Hu, Wenqi Fan, Qing Li, Hongyang Wang, Jun Wang, and Wenwu Ou. 2026. A/B Agent: A Self-Evolving Agent for Strategy Iteration in Industrial A/B Testing. arXiv:2608.04625. arXiv:2608.04625 https://arxiv.org/abs/2608.04625

[69] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. 2025. AIDE: AI-Driven Exploration in the Space of Code. arXiv:2502.13138. arXiv:2502.13138 https://arxiv.org/abs/2502.13138

[70] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. 2024. DSBench: How Far Are Data Science Agents from Becoming Data Science Experts? arXiv:2409.07703. arXiv:2409.07703 https://arxiv.org/abs/2409.07703

[71] Ramesh Johari, Leo Pekelis, and David J. Walsh. 2015. Always Valid Inference: Bringing Sequential Analysis to A/B Testing. arXiv:1512.04922. arXiv:1512.04922 https://arxiv.org/abs/1512.04922

[72] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. arXiv:1808.09781 doi:10.1109/icdm.2018.00035

[73] Sayash Kapoor, Emily M. Cantrell, Kenny Peng, Thanh Hien Pham, Christopher A. Bail, Odd Erik Gundersen, Jake M. Hofman, Jessica Hullman, Michael A.

Lones, Momin M. Malik, Priyanka Nanayakkara, Russell A. Poldrack, Inioluwa Deborah Raji, Michael Roberts, Matthew J. Salganik, Marta Serra-Garcia, Brandon M. Stewart, Gilles Vandewiele, and Arvind Narayanan. 2024. REFORMS: Consensus-Based Recommendations for Machine-Learning-Based Science. *Science Advances* 10, 18 (2024), eadk3452. arXiv:2308.07832 doi:10.1126/sciadv.adk3452

[74] Sayash Kapoor and Arvind Narayanan. 2023. Leakage and the Reproducibility Crisis in Machine-Learning-Based Science. *Patterns* 4, 9 (2023), 100804. arXiv:2207.07048 doi:10.1016/j.patter.2023.100804

[75] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2025. AI Agents That Matter. *Transactions on Machine Learning Research* (2025). arXiv:2407.01502 https://openreview.net/forum?id=Zy4uFzMviZ

[76] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations*. arXiv:2310.03714 https://arxiv.org/abs/2310.03714

[77] SungGeun Kim, Abhinav Narain, and Daniel Nemirovsky. 2026. AURA: Agentic Diagnosis and Refinement for Production Recommender Systems at Scale. arXiv:2609.16625. arXiv:2609.16625 https://arxiv.org/abs/2609.16625

[78] Sein Kim, Sangwu Park, Hongseok Kang, Wonjoong Kim, Jimin Seo, Yeonjun In, Kanghoon Yoon, Hyunsik Jeon, and Chanyoung Park. 2026. Self-EvolveRec: Self-Evolving Recommender Systems with LLM-based Directional Feedback. arXiv:2602.12612. arXiv:2602.12612 https://arxiv.org/abs/2602.12612

[79] Anton Klenitskiy and Alexey Vasilev. 2023. Turning Dross Into Gold Loss: is BERT4Rec really better than SASRec?. In *Proceedings of the 17th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 1120–1125. arXiv:2309.07602 doi:10.1145/3604915.3610644

[80] Ron Kohavi, Diane Tang, and Ya Xu. 2020. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press. doi:10.1017/9781108653985

[81] Patrick Tser Jern Kon, Jiachen Liu, Xinyi Zhu, Qiuyi Ding, Jingjia Peng, Jiarong Xing, Yibo Huang, Yiming Qiu, Jayanth Srinivasa, Myungjin Lee, Mosharaf Chowdhury, Matei Zaharia, and Ang Chen. 2025. EXP-Bench: Can AI Conduct AI Research Experiments? arXiv:2505.24785. arXiv:2505.24785 https://arxiv.org/abs/2505.24785

[82] Walid Krichene and Steffen Rendle. 2020. On Sampled Metrics for Item Recommendation. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, New York, NY, USA, 1748–1757. doi:10.1145/3394486.3403226

[83] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. 2025. ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution. arXiv:2509.19349. arXiv:2509.19349 https://arxiv.org/abs/2509.19349

[84] Changxin Lao, Fei Pan, Guozhuang Ma, Han Li, Huihuang Lin, Jijun Shi, Kangzhi Zhao, Kun Gai, Mo Zhou, Qinqin Zhou, Quan Chen, Ruochen Yang, Shifu Bie, Shijie Yi, Shuang Yang, Shuo Yang, Wenhao Li, Wentao Xie, Xiao Lv, Xuming Wang, Yijun Wang, Yiming Chen, Yusheng Huang, Zhongyuan Wang, Zibo Zhao, Zijie Zhuang, Baoning Xia, Chao Liu, Chaoyi Ma, Chubo He, Dawei Cong, Feng Jiang, Gang Wang, Guilin Xia, Hanwen Xu, Jiahong Xie, Jiahui Qiao, Jian Liang, Jiangfan Yue, Jing Wang, Jinghan Yang, Jinghui Jia, Kan Qin, Lei Wang, Ming Li, Peilin Song, Pengbo Xu, Qiang Luo, Ruiming Tang, Shiyang Liu, Shuxian Jin, Tao Wang, Tao Zhang, Xiang Gao, Xianghan Li, Yingsong Luo, Yiwen Ning, Yongcheng Liu, Yueyang Liu, Yuan Guo, Zhaojie Liu, and Zhenkai Cui. 2026. AgentX: Towards Agent-Driven Self-Iteration of Industrial Recommender Systems. arXiv:2606.26859. arXiv:2606.26859 https://arxiv.org/abs/2606.26859

[85] Minyong R. Lee and Milan Shen. 2018. Winner's Curse: Bias Estimation for Total Effects of Features in Online Controlled Experiments. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, New York, NY, USA, 491–499. doi:10.1145/3219819.3219905

[86] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-Harness: End-to-End Optimization of Model Harnesses. arXiv:2603.28052. arXiv:2603.28052 https://arxiv.org/abs/2603.28052

[87] Bingqian Li, Xiaolei Wang, Junyi Li, Weitao Li, Long Zhang, Sheng Chen, Wayne Xin Zhao, and Ji-Rong Wen. 2026. RecNet: Self-Evolving Preference Propagation for Agentic Recommender Systems. arXiv:2601.21609. arXiv:2601.21609 https://arxiv.org/abs/2601.21609

[88] Lihong Li, Wei Chu, John Langford, and Xuanhui Wang. 2011. Unbiased Offline Evaluation of Contextual-bandit-based News Article Recommendation Algorithms. In *Proceedings of the Fourth ACM International Conference on Web Search and Data Mining*. ACM, New York, NY, USA, 297–306. arXiv:1003.5956 doi:10.1145/1935826.1935878

[89] Liam Li and Ameet Talwalkar. 2020. Random Search and Reproducibility for Neural Architecture Search. In *Proceedings of the 35th Uncertainty in Artificial Intelligence Conference (UAI) (Proceedings of Machine Learning Research, Vol. 115)*. PMLR, 367–377. arXiv:1902.07638 https://proceedings.mlr.press/v115/li20c.html

[90] Ming Li, Dai Li, Xuying Ning, Bo Sun, Rui Li, Yi Zhang, Silvia Gong, Xuan Cao, Rui Li, Cornelia Carapcea, Qunshu Zhang, Zhigang Wang, Yinglong Xia, and Andy Wang. 2026. Auto-RecSys: Harnessing Autonomous Research Agents for Industry-Scale Recommender Systems. arXiv:2609.10922. arXiv:2609.10922 https://arxiv.org/abs/2609.10922

[91] Matthew Li, Albert J. Liang, and Dilip Guru Raj. 2026. MLE Agents: Autonomous ML Engineering in Production Recommendation Systems. In *NeurIPS 2026 Workshop on Autonomous Machine Learning Research*. https://openreview.net/forum?id=b2UcmSJTPK

[92] Xiaopeng Li, Kuo Cai, Bo Chen, Wenlin Zhang, Mengyang Ma, Yingyi Zhang, Zichuan Fu, Yu Yang, Qidong Liu, Yiyu Wang, Ruiming Tang, Wenwu Ou, Jiang Wu, Zhanbo Xu, and Xiangyu Zhao. 2026. EvoSkillRec: Skill-Genome Evolution for Recommender Architecture Discovery. arXiv:2609.34552. arXiv:2609.34552 https://arxiv.org/abs/2609.34552

[93] Zelong Li, Jianchao Ji, Yingqiang Ge, and Yongfeng Zhang. 2022. AutoLossGen: Automatic Loss Function Generation for Recommender Systems. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 1304–1315. arXiv:2204.13160 doi:10.1145/3477495.3531941

[94] Gang Liao, Hongsen Qin, Ying Wang, Alicia Golden, Michael Kuchnik, Yavuz Yetim, Jia Jiunn Ang, Chunli Fu, Yihan He, Samuel Hsia, Zewei Jiang, Dianshi Li, Uladzimir Pashkevich, Varna Puvvada, Feng Shi, Matt Steiner, Ruichao Xiao, Liyuan Li, Nathan Yan, Xiayu Yu, Zhou Fang, Roman Levenstein, Kunming Ho, Haishan Zhu, Alec Hammond, Richard Li, Ajit Mathews, Kaustubh Gondkar, Abdul Zainul-Abedin, Ketan Singh, Hongtao Yu, Wenyuan Chi, Barney Huang, Sean Zhang, Noah Weller, Zach Marine, Wyatt Cook, Carole-Jean Wu, and Gaoxiang Liu. 2025. KernelEvolve: Scaling Agentic Kernel Coding for Heterogeneous AI Accelerators at Meta. arXiv:2512.23236. arXiv:2512.23236 https://arxiv.org/abs/2512.23236

[95] Yuxin Liao, Le Wu, Min Hou, Yu Wang, Han Wu, and Meng Wang. 2026. From Atom to Community: Structured and Evolving Agent Memory for User Behavior Modeling. arXiv:2601.16872. arXiv:2601.16872 https://arxiv.org/abs/2601.16872

[96] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu, Chuhan Wu, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, and Weinan Zhang. 2023. How Can Recommender Systems Benefit from Large Language Models: A Survey. arXiv:2306.05817. arXiv:2306.05817 https://arxiv.org/abs/2306.05817

[97] Jiuning Lin, Ruiquan Lan, Xiaodong Zhu, Bin Zhang, Chengyu Lai, Chuxin Chen, Dimin Wang, Han Zhu, Hongtao Cheng, Jialin Zhu, Lingqing Zhang, Shuai Zhong, Tao Wang, Weipeng Huang, Yinjiang Cai, Yinnan Song, Yuan Liu, Zhibo Xiao, Zhixin Ma, and Zihong Huang. 2026. PILOT Technical Report. arXiv:2608.18637. arXiv:2608.18637 https://arxiv.org/abs/2608.18637

[98] Jiacheng Lin, Tian Wang, and Kun Qian. 2025. Rec-R1: Bridging Generative Large Language Models and User-Centric Recommendation Systems via Reinforcement Learning. *Transactions on Machine Learning Research* (2025). arXiv:2503.24289 https://arxiv.org/abs/2503.24289

[99] Xinyu Lin, Yashar Deldjoo, Sunhao Dai, Honghui Bao, Xiaopeng Ye, Fatemeh Nazary, Wenjie Wang, Tommaso Di Noia, Jun Xu, and Tat-Seng Chua. 2026. Autonomous Information Seeking: A Roadmap for Agentic Recommender Systems. arXiv:2607.04433. arXiv:2607.04433 https://arxiv.org/abs/2607.04433

[100] Xinyu Lin, Zhuosong Jiang, Zixiao Suo, Siqin Wang, Hanqing Zeng, Hanchao Yu, Yinglong Xia, Jiang Zhang, Aashu Singh, Fei Liu, Wenjie Wang, Fuli Feng, Yang Song, Qifan Wang, and Tat-Seng Chua. 2026. Self-Evolving Memory for Generative Recommendation. arXiv:2609.15598. arXiv:2609.15598 https://arxiv.org/abs/2609.15598

[101] Marius Lindauer and Frank Hutter. 2020. Best Practices for Scientific Research on Neural Architecture Search. *Journal of Machine Learning Research* 21, 243 (2020), 1–18. arXiv:1909.02453 https://jmlr.org/papers/v21/20-056.html

[102] Haoran Ling, Yuecheng Li, Zeyu Song, Jing Yao, Shuwen Kang, Chi Lu, Wenjin Wu, and Peng Jiang. 2026. RecHarness: A Bandit-Routed Agentic Harness for Self-Evolving Recommender Systems. arXiv:2607.29241. arXiv:2607.29241 https://arxiv.org/abs/2607.29241

[103] Bochao Liu, Zhipeng Qian, Yang Zhao, Xinyuan Jiang, Zihan Liang, Yufei Ma, Junpeng Zhuang, Ben Chen, Shuo Yang, Hongen Wan, Yao Wu, Chenyi Lei, and Xiao Liang. 2026. Bian Que: An Agentic Framework with Flexible Skill Arrangement for Online System Operations. arXiv:2604.26805. arXiv:2604.26805 https://arxiv.org/abs/2604.26805

[104] Bin Liu, Chenxu Zhu, Guilin Li, Weinan Zhang, Jincai Lai, Ruiming Tang, Xiuqiang He, Zhenguo Li, and Yong Yu. 2020. AutoFIS: Automatic Feature Interaction Selection in Factorization Models for Click-Through Rate Prediction. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, New York, NY, USA, 2636–2645. arXiv:2003.11235 doi:10.1145/3394486.3403314

[105] Dairui Liu, Boming Yang, Honghui Du, Derek Greene, Neil Hurley, Aonghus Lawlor, Ruihai Dong, and Irene Li. 2023. RecPrompt: A Self-tuning Prompting Framework for News Recommendation Using Large Language Models. arXiv:2312.10463. arXiv:2312.10463 https://arxiv.org/abs/2312.10463

[106] Fei Liu, Xinyu Lin, Hanchao Yu, Mingyuan Wu, Jianyu Wang, Qiang Zhang, Zhuokai Zhao, Yinglong Xia, Yao Zhang, Weiwei Li, Mingze Gao, Qifan Wang, Lizhu Zhang, Benyu Zhang, and Xiangjun Fan. 2026. RecoWorld: Building Simulated Environments for Agentic Recommender Systems. In *Companion Proceedings of the ACM Web Conference 2026*. ACM, New York, NY, USA, 650–659. arXiv:2509.10397 doi:10.1145/3774905.3794661

[107] Gang Liu, Yihan Zhu, Jie Chen, and Meng Jiang. 2025. Scientific Algorithm Discovery by Augmenting AlphaEvolve with Deep Research. arXiv:2510.06056. arXiv:2510.06056 https://arxiv.org/abs/2510.06056

[108] Jiahao Liu, Shengkang Gu, Dongsheng Li, Guangping Zhang, Mingzhe Han, Hansu Gu, Peng Zhang, Tun Lu, Li Shang, and Ning Gu. 2025. AgentCF++: Memory-enhanced LLM-based Agents for Popularity-aware Cross-domain Recommendations. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 2566–2571. arXiv:2502.13843 doi:10.1145/3726302.3730161

[109] Shaohua Liu, Liang Fang, Yilong Sun, Shudong Huang, Qingsong Luo, Shaoxin Liu, Xiaoyang Chen, Dongqiang Liu, Chuangang Ma, Zhenzhen Chai, Henghuan Wang, Shijie Quan, Changyuan Cui, Zhangbin Zhu, Peng Chen, Wei Xu, Lei Xiao, Haijie Gu, and Jie Jiang. 2026. NOVA: A Verification-Aware Agent Harness for Architecture Evolution in Industrial Recommender Systems. arXiv:2606.27243. arXiv:2606.27243 https://arxiv.org/abs/2606.27243

[110] Siyi Liu, Chen Gao, and Yong Li. 2024. Large Language Model Agent for Hyper-Parameter Optimization. arXiv:2402.01881. arXiv:2402.01881 https://arxiv.org/abs/2402.01881

[111] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. 2026. AutoHarness: improving LLM agents by automatically synthesizing a code harness. arXiv:2603.03329. arXiv:2603.03329 https://arxiv.org/abs/2603.03329

[112] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv:2408.06292. arXiv:2408.06292 https://arxiv.org/abs/2408.06292

[113] Ziming Luo, Atoosa Kasirzadeh, and Nihar B. Shah. 2025. The More You Automate, the Less You See: Hidden Pitfalls of AI Scientist Systems. arXiv:2509.08713. arXiv:2509.08713 https://arxiv.org/abs/2509.08713

[114] Fuyuan Lyu, Xing Tang, Hong Zhu, Huifeng Guo, Yingxue Zhang, Ruiming Tang, and Xue Liu. 2022. OptEmbed: Learning Optimal Embedding Table for Click-through Rate Prediction. arXiv:2208.04482. arXiv:2208.04482 https://arxiv.org/abs/2208.04482

[115] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Eureka: Human-Level Reward Design via Coding Large Language Models. In *International Conference on Learning Representations*. arXiv:2310.12931 https://arxiv.org/abs/2310.12931

[116] David Manheim and Scott Garrabrant. 2018. Categorizing Variants of Goodhart's Law. arXiv:1803.04585. arXiv:1803.04585 https://arxiv.org/abs/1803.04585

[117] Horia Mania, John Miller, Ludwig Schmidt, Moritz Hardt, and Benjamin Recht. 2019. Model Similarity Mitigates Test Set Overuse. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc. arXiv:1905.12580 https://proceedings.neurips.cc/paper_files/paper/2019/hash/48237d9f2dea8c74c2a72126cf63d933-Abstract.html

[118] Reza Yousefi Maragheh and Yashar Deldjoo. 2025. The Future is Agentic: Definitions, Perspectives, and Open Challenges of Multi-Agent Recommender Systems. arXiv:2507.02097. arXiv:2507.02097 https://arxiv.org/abs/2507.02097

[119] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton van den Hengel. 2015. Image-Based Recommendations on Styles and Substitutes. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 43–52. doi:10.1145/2766462.2767755

[120] Zaiqiao Meng, Richard McCreadie, Craig Macdonald, and Iadh Ounis. 2020. Exploring Data Splitting Strategies for the Evaluation of Recommendation Models. In *Fourteenth ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 681–686. arXiv:2007.13237 doi:10.1145/3383313.3418479

[121] Meta Engineering. 2026. Ranking Engineer Agent (REA): The Autonomous AI Agent Accelerating Meta's Ads Ranking Innovation. Engineering at Meta blog, 17 March 2026. https://engineering.fb.com/2026/03/17/developer-tools/ranking-engineer-agent-rea-autonomous-ai-system-accelerating-meta-ads-ranking-innovation/

[122] METR. 2025. Recent Frontier Models Are Reward Hacking. METR blog, 5 June 2025. https://metr.org/blog/2025-06-05-recent-reward-hacking/

[123] Lingyu Mu, Hao Deng, Haibo Xing, Jinxin Hu, Yu Zhang, and Xiaoyi Zeng. 2026. EvoRec: Self-Evolving Agentic Recommender Systems. arXiv:2606.28368. arXiv:2606.28368 https://arxiv.org/abs/2606.28368

[124] Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan Ö. Arık, and Tomas Pfister. 2025. MLE-STAR: Machine Learning Engineering Agent via Search and Targeted Refinement. In *Advances in Neural Information Processing Systems 38*. 129290–129310. arXiv:2506.15692 doi:10.52202/085713-3889

[125] Bharath Sivaram Narasimhan and Karthik R Narasimhan. 2026. τ-Rec: A Verifiable Benchmark for Agentic Recommender Systems. In *Proceedings of the 20th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 915–919. arXiv:2606.10156 doi:10.1145/3773078.3831847

[126] Muhammad U. Nasir, Sam Earle, Christopher Cleghorn, Steven James, and Julian Togelius. 2023. LLMatic: Neural Architecture Search via Large Language Models and Quality Diversity Optimization. arXiv:2306.01102. arXiv:2306.01102 https://arxiv.org/abs/2306.01102

[127] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. 2025. MLGym: A New Framework and Benchmark for Advancing AI Research Agents. arXiv:2502.14499. arXiv:2502.14499 https://arxiv.org/abs/2502.14499

[128] Minh-Duc Nguyen, Hai-Dang Kieu, and Dung D. Le. 2026. AMEM4Rec: Leveraging Cross-User Similarity for Memory Evolution in Agentic LLM Recommenders. arXiv:2602.08837. arXiv:2602.08837 https://arxiv.org/abs/2602.08837

[129] Jinming Nian, Jyun-Yu Jiang, Henry Tsai, Yi Fang, and Zhirong Liang. 2026. Evolving Semantic Indexes for Generative Retrieval. arXiv:2610.28568. arXiv:2610.28568 https://arxiv.org/abs/2610.28568

[130] Jinming Nian, Fangchen Li, Dae Hoon Park, and Yi Fang. 2026. RankEvolve: Automating the Discovery of Retrieval Algorithms via LLM-Driven Evolution. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 4028–4033. arXiv:2602.16932 doi:10.1145/3805712.3809962

[131] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131. arXiv:2506.13131 https://arxiv.org/abs/2506.13131

[132] OpenEvolve contributors. 2025. OpenEvolve: An Open-Source Implementation of AlphaEvolve. https://github.com/algorithmicsuperintelligence/openevolve.

[133] Kesha Ou, Chenghao Wu, Xiaolei Wang, Bowen Zheng, Wayne Xin Zhao, Weitao Li, Long Zhang, Sheng Chen, and Ji-Rong Wen. 2026. Deep Research for Recommender Systems. arXiv:2603.07605. arXiv:2603.07605 https://arxiv.org/abs/2603.07605

[134] Enrico Palumbo, Alexandre Tamborrino, Victor Ode, Ben Lacker, Adrià Casas Escoda, Jeremy Hopple, Marcus Better, James Leoni, Hugo Galvão, Hugues Bouchard, Mounia Lalmas, José Luis Redondo García, Abenezer Abebe, Ann Clifton, Anton Blomberg, Henrik Lindström, Dani Doro, and Christine Doig Cardet. 2026. Bootstrapping Conversational Recommendation Agents At Spotify: Synthetic Data Generation and Self-Improvement Loops. In *Proceedings of the 20th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 994–1003. arXiv:2609.30297 doi:10.1145/3773078.3831910

[135] Weidi Pan, He Ma, Shuhao Ye, Palaksh Rungta, David McPeek, Junyi Jiao, Arnab Bhadury, Mingyan Gao, and Onkar Dalal. 2026. RecEvolve: A Knowledge-Driven Autonomous Agent System for Recommender Systems. In *Proceedings of the 20th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 1437–1442. arXiv:2609.01622 doi:10.1145/3773078.3831898 The audit coded the arXiv version.

[136] Rayhan Patel and Shabaz Patel. 2026. EvoRank: LLM-Guided Evolution of Multi-Objective Learning-to-Rank Pipelines. arXiv:2609.22196. arXiv:2609.22196 https://arxiv.org/abs/2609.22196

[137] Bo Peng, Lizhu Zhang, Yuhang Zhou, Mingyi Wang, Yifan Wu, Serena Li, Xiangjun Fan, and Zhuokai Zhao. 2026. Discovered, Not Designed: Population Evolution for Collaborative and Compute-Intensive Model Discovery. arXiv:2610.05950. arXiv:2610.05950 https://arxiv.org/abs/2610.05950

[138] Qiyao Peng, Hongtao Liu, Hua Huang, Jian Yang, Qing Yang, and Minglai Shao. 2025. A Survey on LLM-powered Agents for Recommender Systems. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. 11574–11583. arXiv:2502.10050 doi:10.18653/v1/2025.findings-emnlp.620

[139] Aleksandr Petrov and Craig Macdonald. 2022. A Systematic Review and Replicability Study of BERT4Rec for Sequential Recommendation. In *Proceedings of the 16th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 436–447. arXiv:2207.07483 doi:10.1145/3523227.3548487

[140] Rushi Qiang, Yuchen Zhuang, Yinghao Li, Dingu Sagar V K, Rongzhi Zhang, Changhao Li, Ian Shu-Hei Wong, Sherry Yang, Percy Liang, Chao Zhang, and Bo Dai. 2025. MLE-Dojo: Interactive Environments for Empowering LLM Agents in Machine Learning Engineering. arXiv:2505.07782. arXiv:2505.07782 https://arxiv.org/abs/2505.07782

[141] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Advances in Neural Information Processing Systems 36*. arXiv:2305.05065 https://arxiv.org/abs/2305.05065

[142] Esteban Real, Chen Liang, David R. So, and Quoc V. Le. 2020. AutoML-Zero: Evolving Machine Learning Algorithms From Scratch. In *Proceedings of the 37th International Conference on Machine Learning*. arXiv:2003.03384 https://arxiv.org/abs/2003.03384

[143] recforge contributors. 2026. recforge: One Protocol to Benchmark and Evolve Recommenders. https://github.com/shangliy/recforge.

[144] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. 2019. Do ImageNet Classifiers Generalize to ImageNet?. In *Proceedings of the 36th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 5389–5400. arXiv:1902.10811 https://proceedings.mlr.press/v97/recht19a.html

[145] Steffen Rendle. 2019. Evaluation Metrics for Item Recommendation under Sampling. arXiv:1912.02263. arXiv:1912.02263 https://arxiv.org/abs/1912.02263

[146] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian Personalized Ranking from Implicit Feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*. 452–461. arXiv:1205.2618

[147] Steffen Rendle, Walid Krichene, Li Zhang, and John Anderson. 2020. Neural Collaborative Filtering vs. Matrix Factorization Revisited. In *Fourteenth ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 240–248. arXiv:2005.09683 doi:10.1145/3383313.3412488

[148] Carson Rodrigues, Oysturn Vas, Isaiah Abner DCosta, and Nithish Kumar Prabhakaran. 2026. When Is an LLM Worth It for Hyperparameter Optimization? A Budget-Matched Study on Tabular Data Finds the Warm-Start Is a Default Configuration, Not the Model. arXiv:2606.21641. arXiv:2606.21641 https://arxiv.org/abs/2606.21641

[149] Rebecca Roelofs, Vaishaal Shankar, Benjamin Recht, Sara Fridovich-Keil, Moritz Hardt, John Miller, and Ludwig Schmidt. 2019. A Meta-Analysis of Overfitting in Machine Learning. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/hash/ee39e503b6bedf0c98c388b7e8589aca-Abstract.html

[150] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* 625, 7995 (2024), 468–475. doi:10.1038/s41586-023-06924-6

[151] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*. ACM, 285–295. doi:10.1145/371920.372071

[152] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. 2025. Agent Laboratory: Using LLM Agents as Research Assistants. arXiv:2501.04227. arXiv:2501.04227 https://arxiv.org/abs/2501.04227

[153] Faisal Shehzad and Dietmar Jannach. 2023. Everyone's a Winner! On Hyperparameter Tuning of Recommendation Models. In *Proceedings of the 17th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 652–657. doi:10.1145/3604915.3609488

[154] Xiang Shen, Yuhang Zhou, Yifan Wu, Zhuokai Zhao, Siyu Lin, Lei Huang, Qianqian Zhong, Lizhu Zhang, Benyu Zhang, Xiangjun Fan, and Hong Yan. 2026. Agentic Recommender System with Hierarchical Belief-State Memory. arXiv:2605.14401. arXiv:2605.14401 https://arxiv.org/abs/2605.14401

[155] Zhan Shi and Shanglin Yang. 2026. Selective LLM-Guided Regularization for Enhancing Recommendation Models. In *Companion Proceedings of the 19th ACM International Conference on Web Search and Data Mining*. 148–153. doi:10.1145/3779211.3795736 GenAIRecP Workshop; arXiv:2512.21526.

[156] Parshin Shojaee, Kazem Meidani, Shashank Gupta, Amir Barati Farimani, and Chandan K Reddy. 2025. LLM-SR: Scientific Equation Discovery via Programming with Large Language Models. In *International Conference on Learning Representations*. arXiv:2404.18400 https://arxiv.org/abs/2404.18400

[157] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. 2022. Defining and Characterizing Reward Hacking. In *Advances in Neural Information Processing Systems*, Vol. 35. Curran Associates, Inc., 9460–9471. arXiv:2209.13085 doi:10.52202/068431-0687

[158] Blaž Škrlj, Benoît Guilleminot, and Andraž Tori. 2025. Agent0: Leveraging LLM Agents to Discover Multi-value Features from Text for Enhanced Recommendations. arXiv:2507.18993. arXiv:2507.18993 https://arxiv.org/abs/2507.18993

[159] Qingquan Song, Dehua Cheng, Hanning Zhou, Jiyan Yang, Yuandong Tian, and Xia Hu. 2020. Towards Automated Neural Interaction Discovery for Click-Through Rate Prediction. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, New York, NY, USA, 945–955. arXiv:2007.06434 doi:10.1145/3394486.3403137

[160] Dhruv Srikanth, Bingchen Zhao, Dixing Xu, Yuxiang Wu, and Zhengyao Jiang. 2026. Recursive self-improvement of AI research agents. arXiv:2609.26457. arXiv:2609.26457 https://arxiv.org/abs/2609.26457

[161] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. 2025. PaperBench: Evaluating AI's Ability to Replicate AI Research. In *Proceedings of the 42nd International Conference on Machine Learning*. arXiv:2504.01848 https://arxiv.org/abs/2504.01848

[162] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. arXiv:1904.06690. arXiv:1904.06690 https://arxiv.org/abs/1904.06690

[163] Zhu Sun, Hui Fang, Jie Yang, Xinghua Qu, Hongyang Liu, Di Yu, Yew-Soon Ong, and Jie Zhang. 2023. DaisyRec 2.0: Benchmarking Recommendation for Rigorous Evaluation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 7 (2023), 8206–8226. arXiv:2206.10848 doi:10.1109/tpami.2022.3231891

[164] Zhu Sun, Di Yu, Hui Fang, Jie Yang, Xinghua Qu, Jie Zhang, and Cong Geng. 2020. Are We Evaluating Rigorously? Benchmarking Recommendation for Reproducible Evaluation and Fair Comparison. In *Proceedings of the 14th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 23–32. doi:10.1145/3383313.3412489

[165] Zhen Tao, Riwei Lai, Chenyun Yu, Weixin Chen, Li Chen, Beibei Kong, Lei Cheng, Chengxiang Zhuo, Zang Li, and Qingqiang Sun. 2026. SAGER: Self-Evolving User Policy Skills for Recommendation Agent. arXiv:2604.14972. arXiv:2604.14972 https://arxiv.org/abs/2604.14972

[166] Himanshu Thakur, Anusha Kamath, Anurag Muthyala, Dhwani Sanmukhani, Smruthi Mukund, and Jay Katukuri. 2026. Towards Reliable ML Feature Engineering via Planning in Constrained-Topology of LLM Agents. arXiv:2601.10820. arXiv:2601.10820 https://arxiv.org/abs/2601.10820

[167] Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Rishi Hazra, Nicolas Baldwin, Alexis Audran-Reiss, Michael Kuchnik, Despoina Magka, Minqi Jiang, Alisia Maria Lupidi, Andrei Lupu, Roberta Raileanu, Kelvin Niu, Tatiana Shavrina, Jean-Christophe Gagnon-Audet, Michael Shvartsman, Shagun Sodhani, Alexander H. Miller, Abhishek Charnalia, Derek Dunfield, Carole-Jean Wu, Pontus Stenetorp, Nicola Cancedda, Jakob Nicolaus Foerster, and Yoram Bachrach. 2025. AI Research Agents for Machine Learning: Search, Exploration, and Generalization in MLE-bench. In *Advances in Neural Information Processing Systems 38*. 39717–39756. arXiv:2507.02554 doi:10.52202/085713-1186

[168] Patara Trirat, Wonyong Jeong, and Sung Ju Hwang. 2024. AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML. arXiv:2410.02958. arXiv:2410.02958 https://arxiv.org/abs/2410.02958

[169] Chenglong Wang, Canjia Li, Xingzhao Zhu, Yifu Huo, Huiyu Wang, Weixiong Lin, Yun Yang, Qiaozhi He, Tianhua Zhou, Xiaojia Chang, Jingbo Zhu, and Tong Xiao. 2026. SERM: Self-Evolving Relevance Model with Agent-Driven Learning from Massive Query Streams. arXiv:2601.09515. arXiv:2601.09515 https://arxiv.org/abs/2601.09515

[170] Haochen Wang, Yi Wu, Daryl Chang, Li Wei, and Lukasz Heldt. 2026. Self-Evolving Recommendation System: End-To-End Autonomous Model Optimization With LLM Agents. In *Proceedings of the 20th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 1462–1470. arXiv:2602.10226 doi:10.1145/3773078.3831919 The audit coded the arXiv version.

[171] Lei Wang, Jingsen Zhang, Hao Yang, Zhi-Yuan Chen, Jiakai Tang, Zeyu Zhang, Xu Chen, Yankai Lin, Hao Sun, Ruihua Song, Wayne Xin Zhao, Jun Xu, Zhicheng Dou, Jun Wang, and Ji-Rong Wen. 2025. User Behavior Simulation with Large Language Model-based Agents. *ACM Transactions on Information Systems* 43, 2 (2025), 1–37. arXiv:2306.02552 doi:10.1145/3708985

[172] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural Graph Collaborative Filtering. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 165–174. doi:10.1145/3331184.3331267

[173] Yifan Wang, Shipeng Zhu, Fei Xiong, Yuqin Yang, Yonghui Huang, Kunyao Wu, Yue Wang, Weichao Meng, and Yu Gong. 2026. PEAR: Progressive Evidence-Based AutoResearch for Industrial Search Systems. arXiv:2609.35031. arXiv:2609.35031 https://arxiv.org/abs/2609.35031

[174] Ziyan Wang, Yingpeng Du, Zhu Sun, Haoyan Chua, Kaidong Feng, Wenya Wang, and Jie Zhang. 2024. Re2LLM: Reflective Reinforcement Large Language Model for Session-based Recommendation. arXiv:2403.16427. arXiv:2403.16427 https://arxiv.org/abs/2403.16427

[175] Zongwei Wang, Min Gao, Hongzhi Yin, Junliang Yu, Tong Chen, Quoc Viet Hung Nguyen, Shazia Sadiq, and Tianrui Li. 2026. Self-Distilled Reinforcement Learning for Co-Evolving Agentic Recommender Systems. arXiv:2604.10029. arXiv:2604.10029 https://arxiv.org/abs/2604.10029

[176] Zongwei Wang, Min Gao, Junliang Yu, Yupeng Hou, Shazia Sadiq, and Hongzhi Yin. 2025. RuleAgent: Discovering Rules for Recommendation Denoising with Autonomous Language Agents. arXiv:2503.23374. arXiv:2503.23374 https://arxiv.org/abs/2503.23374

[177] Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Holden Karnofsky, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. 2024. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. arXiv:2411.15114. arXiv:2411.15114 https://arxiv.org/abs/2411.15114

[178] Sherman Wong, Zhenting Qi, Zhaodong Wang, Nathan Hu, Samuel Lin, Jun Ge, Erwin Gao, Wenlin Chen, Yilun Du, Minlan Yu, and Ying Zhang. 2025. Confucius Code Agent: Scalable Agent Scaffolding for Real-World Codebases.

arXiv:2512.10398. arXiv:2512.10398 https://arxiv.org/abs/2512.10398

[179] Chaowei Wu, Huazhu Chen, Congde Yuan, Qirui Yang, Guoqing Song, Yue Gao, Li Luo, Frank Youhua Chen, and Mengzhuo Guo. 2026. AgentLTV: An Agent-Based Unified Search-and-Evolution Framework for Automated Lifetime Value Prediction. arXiv:2602.21634. arXiv:2602.21634 https://arxiv.org/abs/2602.21634

[180] Fangzhao Wu, Ying Qiao, Jiun-Hung Chen, Chuhan Wu, Tao Qi, Jianxun Lian, Danyang Liu, Xing Xie, Jianfeng Gao, Winnie Wu, and Ming Zhou. 2020. MIND: A Large-scale Dataset for News Recommendation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 3597–3606. doi:10.18653/v1/2020.acl-main.331

[181] Jie JW Wu, Ayanda Patrick Herlihy, Ahmad Saleem Mirza, Ali Afoud, and Fatemeh Fard. 2025. GrowthHacker: Automated Off-Policy Evaluation Optimization Using Code-Modifying LLM Agents. arXiv:2511.00802. arXiv:2511.00802 https://arxiv.org/abs/2511.00802

[182] Liwei Wu and Cho-Jui Hsieh. 2026. Closing the Auto-Research Loop: An AI Co-Scientist for Production Search Ranking. arXiv:2603.22376. arXiv:2603.22376 https://arxiv.org/abs/2603.22376

[183] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, Hui Xiong, and Enhong Chen. 2023. A Survey on Large Language Models for Recommendation. arXiv:2305.19860. arXiv:2305.19860 https://arxiv.org/abs/2305.19860

[184] Xidong Wu, Yue Zhuan, Ruoqiao Wei, Hangxin Chen, Di Bai, Jintao Liu, Xinyi Wang, Xue Wang, Luoshu Wang, and Xinwu Cheng. 2026. AgenticRecTune: Multi-Agent with Self-Evolving Skillhub for Recommendation System Optimization. arXiv:2604.26969. arXiv:2604.26969 https://arxiv.org/abs/2604.26969

[185] Zhouhang Xie, Bo Peng, Zhankui He, Ziqi Chen, Alice Han, Isabella Ye, Benjamin Coleman, Noveen Sachdeva, Fernando Pereira, Julian McAuley, Wang-Cheng Kang, Derek Zhiyuan Cheng, Beidou Wang, and Randolph Brown. 2026. AgenticTagger: Structured Item Representation for Recommendation with LLM Agents. arXiv:2602.05945. arXiv:2602.05945 https://arxiv.org/abs/2602.05945

[186] Jinfeng Xu, Zheyu Chen, Ziyue Peng, Zheng Lin, Wenhao Yuan, Jian Chen, Shujie Li, and Edith Ngai. 2026. Beyond Successor Accuracy: State Retention for Recursive Self-Improvement in Recommendation. arXiv:2610.07105. arXiv:2610.07105 https://arxiv.org/abs/2610.07105

[187] Ruiyao Xu, Yanfei Chen, Zhongying CuiZhu, Bhavana Dalvi Mishra, Yifei Ming, Han Yu, Rujun Han, Chen-Yu Lee, and Tomas Pfister. 2026. Harness^2: Recursive Agent Harnessing for an Open World. arXiv:2610.14224. arXiv:2610.14224 https://arxiv.org/abs/2610.14224

[188] Yiqiao Xu, Yunqi Mi, Yingjie Wu, et al. 2026. From Scoring to Workflow: A Survey of Agentic Recommender Systems. *Information Processing & Management* 64, 2 (2026), 105164. doi:10.1016/j.ipm.2026.105164

[189] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. 2025. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. arXiv:2504.08066. arXiv:2504.08066 https://arxiv.org/abs/2504.08066

[190] Antoine Yang, Pedro M. Esperança, and Fabio M. Carlucci. 2020. NAS Evaluation is Frustratingly Hard. In *International Conference on Learning Representations (ICLR)*. arXiv:1912.12522 https://arxiv.org/abs/1912.12522

[191] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. In *International Conference on Learning Representations*. arXiv:2309.03409 https://arxiv.org/abs/2309.03409

[192] Hanchen Yang, Kaiwen Yang, Junpeng Zhuang, Yang He, Keting Cen, Bochao Liu, Zhongbo Sun, An Liu, Zhongteng Han, and Chenyi Lei. 2026. SR-Agent: An Experience-Driven Agentic Framework for Post-Ranking Strategy Refinement in E-Commerce Recommendation. arXiv:2607.17719. arXiv:2607.17719 https://arxiv.org/abs/2607.17719

[193] Shanglin Yang. 2026. ideation-goodhart: What Should You Optimize When You Optimize an LLM Research-Ideation Harness? https://github.com/shangliy/ideation-goodhart. Code, pre-registrations and report.

[194] Shuang Yang, Zijie Zhuang, Changxin Lao, Pengbo Xu, Hanwen Xu, Yusheng Huang, Han Gao, Guanchen Wang, Tianbao Ma, Linxun Chen, Peilin Song, Xuming Wang, Chen Li, Fan Wu, Tao Wang, Zibo Zhao, Xiangyu Wu, An Liu, Fei Pan, Peng Jiang, Chen Yang, Zhaojie Liu, and Wenwu Ou. 2026. Advancing Model Research in AgentX: Long-Horizon Autonomy for Industrial Recommender Systems. arXiv:2609.30001. arXiv:2609.30001 https://arxiv.org/abs/2609.30001

[195] Xu Yang, Xiao Yang, Shikai Fang, Yifei Zhang, Jian Wang, Bowen Xian, Qizheng Li, Jingyuan Li, Minrui Xu, Yuante Li, Haoran Pan, Yuge Zhang, Weiqing Liu, Yelong Shen, Weizhu Chen, and Jiang Bian. 2025. R&D-Agent: An LLM-Agent Framework Towards Autonomous Data Science. arXiv:2505.14738. arXiv:2505.14738 https://arxiv.org/abs/2505.14738

[196] Kehui Yao, Chuanbo Hu, Jiao Chen, Ramin Giahi, Reza Yousefi Maragheh, Lalitesh Morishetti, Ziwei Hong, Yiyun Su, Jianpeng Xu, Evren Korpeoglu, and Kannan Achan. 2026. An Explicit Feedback Contract for Iterative ML-Agent Harnesses. arXiv:2610.28798. arXiv:2610.28798 https://arxiv.org/abs/2610.28798

[197] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. 2024. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. In *Advances in Neural Information Processing Systems 37*. arXiv:2402.01145 https://arxiv.org/abs/2402.01145

[198] Chris Ying, Aaron Klein, Eric Christiansen, Esteban Real, Kevin Murphy, and Frank Hutter. 2019. NAS-Bench-101: Towards Reproducible Neural Architecture Search. In *Proceedings of the 36th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 7105–7114. arXiv:1902.09635 https://proceedings.mlr.press/v97/ying19a.html

[199] Caiyang Yu, Xianggen Liu, Yifan Wang, Yun Liu, Wentao Feng, Deng Xiong, Chenwei Tang, and Jiancheng Lv. 2023. GPT-NAS: Evolutionary Neural Architecture Search with the Generative Pre-Trained Model. arXiv:2305.05351. arXiv:2305.05351 https://arxiv.org/abs/2305.05351

[200] Kaicheng Yu, Christian Sciuto, Martin Jaggi, Claudiu Musat, and Mathieu Salzmann. 2020. Evaluating the Search Phase of Neural Architecture Search. In *International Conference on Learning Representations (ICLR)*. arXiv:1902.08142 https://arxiv.org/abs/1902.08142

[201] Zheng Yuan, Fajie Yuan, Yu Song, Youhua Li, Junchen Fu, Fei Yang, Yunzhu Pan, and Yongxin Ni. 2023. Where to Go Next for Recommender Systems? ID- vs. Modality-based Recommender Models Revisited. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2639–2649. doi:10.1145/3539618.3591932

[202] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. TextGrad: Automatic "Differentiation" via Text. arXiv:2406.07496. arXiv:2406.07496 https://arxiv.org/abs/2406.07496

[203] Longfei Yun, Yihan Wu, Haoran Liu, Xiaoxuan Liu, Ziyun Xu, Yi Wang, Yang Xia, Pengfei Wang, Mingze Gao, Yunxiang Wang, Changfan Chen, Fei Liu, Akos Lada, Wenjie Fu, Hong Yan, and Junfeng Pan. 2026. Decoding ML Decision: An Agentic Reasoning Framework for Large-Scale Ranking System. arXiv:2602.18640. arXiv:2602.18640 https://arxiv.org/abs/2602.18640

[204] Wenwen Zeng, Jinhui Zhang, Hao Chen, Zhaoyu Hu, Yongqi Liang, Jiajun Chai, Dengcan Liu, Zhenfeng Liu, Shurui Yan, Minglong Xue, Xiaohan Wang, Wei Lin, and Guojun Yin. 2026. RecRM-Bench: Benchmarking Multidimensional Reward Modeling for Agentic Recommender Systems. arXiv:2605.11874. arXiv:2605.11874 https://arxiv.org/abs/2605.11874

[205] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR 235*. arXiv:2402.17152 https://arxiv.org/abs/2402.17152

[206] An Zhang, Yuxin Chen, Leheng Sheng, Xiang Wang, and Tat-Seng Chua. 2024. On Generative Agents in Recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 1807–1817. arXiv:2310.10108 doi:10.1145/3626772.3657844

[207] Bin Zhang, Bowen Zheng, Chao Yi, Chengyu Lai, Dian Chen, Dimin Wang, Gaoyang Guo, Jialin Zhu, Jian Wu, Jing Yu, Jiuning Lin, Lingqing Zhang, Lingyun Zheng, Mao Zhang, Mingming Pan, Ruiquan Lan, Shuai Zhong, Wen Chen, Wendong Zhang, Xiaodong Zhu, Xuan Chen, Xunke Xi, Yifan Lu, Yiheng Wang, Yue Zeng, Yujie Luo, Yuning Jiang, Zhe Hu, Zhibo Xiao, Zihong Huang, Binbin Cao, Bo Zheng, Danning Wang, Dixuan Wang, Ge Fan, Haixia Wu, Han Zhu, Hao Fang, Haoming Chen, Huiping Chu, Jian Wang, Jianjun Wu, Jiawei Wu, Jiaxin Yu, Jingwen Liu, Jinzhe Shan, Kai Meng, Kai Zhang, Keqin Xu, Kewei Zhu, Lang Tian, Leihui Chen, Li Chen, Licheng Xu, Lide Xiao, Ruitong Zhang, Shiyao Peng, Silu Zhou, Tao Wang, Wei Shi, Wenjun Yang, Xiang Chen, Xiang Gao, Xiao Ren, Xu Liu, Xuwen Wang, Yang Li, Yeqiu Yang, Yi Hu, Yichen Yuan, Yinnan Song, Yipeng Yu, Yuan Liu, Yunqi Gao, Zhiliang Huang, Zhujin Gao, and Zongyuan Wu. 2026. DREAM Technical Report. arXiv:2608.09408. arXiv:2608.09408 https://arxiv.org/abs/2608.09408

[208] Hao Zhang, Jinxin Hu, Hao Deng, Lingyu Mu, Shizhun Wang, Yu Zhang, and Xiaoyi Zeng. 2026. AgenticRS-Architecture: System Design for Agentic Recommender Systems. arXiv:2603.26085. arXiv:2603.26085 https://arxiv.org/abs/2603.26085

[209] Junjie Zhang, Yupeng Hou, Ruobing Xie, Wenqi Sun, Julian McAuley, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2024. AgentCF: Collaborative Learning with Autonomous Language Agents for Recommender Systems. In *Proceedings of the ACM Web Conference 2024*. ACM, New York, NY, USA, 3679–3689. arXiv:2310.09233 doi:10.1145/3589334.3645537

[210] Luankang Zhang, Hao Wang, Zhongzhou Liu, Mingjia Yin, Yonghao Huang, Jiaqi Li, Wei Guo, Yong Liu, Huifeng Guo, Defu Lian, and Enhong Chen. 2026. Can Recommender Systems Teach Themselves? A Recursive Self-Improving Framework with Fidelity Control. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. arXiv:2602.15659 https://arxiv.org/abs/2602.15659

[211] Liulei Zhang, Dejing Zhou, Chuyue Huang, Guanhua Chen, Yutong Yao, Lidia S. Chao, Chi Man Vong, and Derek F. Wong. 2026. How Is Automated Research Evaluated? A Survey of Benchmarks and Evaluation Practices. arXiv:2610.11877. arXiv:2610.11877 https://arxiv.org/abs/2610.11877

[212] Michael R. Zhang, Nishkrit Desai, Juhan Bae, Jonathan Lorraine, and Jimmy Ba. 2023. Using Large Language Models for Hyperparameter Optimization.

arXiv:2312.04528. arXiv:2312.04528 https://arxiv.org/abs/2312.04528

[213] Qi Zhang, Yanlin Chen, and Wenchao Xiao. 2026. AutoLR: Automating the Path from Research to Launch Review in Industrial Recommender Systems. arXiv:2609.04871. arXiv:2609.04871 https://arxiv.org/abs/2609.04871

[214] Sheng Zhang, Maolin Wang, Yao Zhao, Chenyi Zhuang, Jinjie Gu, Ruocheng Guo, Xiangyu Zhao, Zijian Zhang, and Hongzhi Yin. 2024. DNS-Rec: Data-aware Neural Architecture Search for Recommender Systems. In *18th ACM Conference on Recommender Systems*. ACM, New York, NY, USA, 591–600. arXiv:2402.00390 doi:10.1145/3640457.3688117

[215] Tunhou Zhang, Dehua Cheng, Yuchen He, Zhengxing Chen, Xiaoliang Dai, Liang Xiong, Feng Yan, Hai Li, Yiran Chen, and Wei Wen. 2023. NASRec: Weight Sharing Neural Architecture Search for Recommender Systems. In *Proceedings of the ACM Web Conference 2023*. ACM, New York, NY, USA, 1199–1207. arXiv:2207.07187 doi:10.1145/3543507.3583446

[216] Wei Zhang, Hongji Li, Song Sun, Peng Yu, Xue Yang, Lei Zhao, and Peng Jiang. 2026. DMRL: Document-Mediated Reinforcement Learning for Skill Optimization in Advertising Recommendation. arXiv:2609.02170. arXiv:2609.02170 https://arxiv.org/abs/2609.02170

[217] Yunxiang Zhang, Muhammad Khalifa, Shitanshu Bhushan, Grant D Murphy, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. 2025. MLRC-Bench: Can Language Agents Solve Machine Learning Research Challenges?. In *Advances in Neural Information Processing Systems 38*. 100801–100845. arXiv:2504.09702 doi:10.52202/085713-3036

[218] Yu Zhang, Shutong Qiao, Jiaqi Zhang, Tzu-Heng Lin, Chen Gao, and Yong Li. 2026. A Survey of Large Language Model Empowered Agents for Recommendation and Search: Towards Next-Generation Information Retrieval. *ACM Transactions on Information Systems*, Article 3842663 (2026). arXiv:2503.05659 doi:10.1145/3842663

[219] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, Yingqian Min, Zhichao Feng, Xinyan Fan, Xu Chen, Pengfei Wang, Wendi Ji, Yaliang Li, Xiaoling Wang, and Ji-Rong Wen. 2021. RecBole: Towards a Unified, Comprehensive and Efficient Framework for Recommendation Algorithms. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. ACM, New York, NY, USA, 4653–4664. arXiv:2011.01731 doi:10.1145/3459637.3482016

[220] Xiangyu Zhao, Haochen Liu, Wenqi Fan, Hui Liu, Jiliang Tang, Chong Wang, Ming Chen, Xudong Zheng, Xiaobing Liu, and Xiwang Yang. 2021. AutoEmb: Automated Embedding Dimensionality Search in Streaming Recommendations. In *2021 IEEE International Conference on Data Mining (ICDM)*. 896–905. arXiv:2002.11252 doi:10.1109/icdm51629.2021.00101

[221] Xiangyu Zhao, Haochen Liu, Hui Liu, Jiliang Tang, Weiwei Guo, Jun Shi, Sida Wang, Huiji Gao, and Bo Long. 2020. Memory-efficient Embedding for Recommendations. arXiv:2006.14827. arXiv:2006.14827 https://arxiv.org/abs/2006.14827

[222] Mingkai Zheng, Xiu Su, Shan You, Fei Wang, Chen Qian, Chang Xu, and Samuel Albanie. 2023. Can GPT-4 Perform Neural Architecture Search? arXiv:2304.10970. arXiv:2304.10970 https://arxiv.org/abs/2304.10970

[223] Ruiqi Zheng, Liang Qu, Bin Cui, Yuhui Shi, and Hongzhi Yin. 2023. AutoML for Deep Recommender Systems: A Survey. *ACM Transactions on Information Systems*, Article 3579355 (2023). arXiv:2203.13922 doi:10.1145/3579355

[224] Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. 2026. ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases. In *International Conference on Learning Representations (ICLR)*. arXiv:2510.20270 https://openreview.net/forum?id=SeO4vyAj7E

[225] Guorui Zhou, Honghui Bao, Jiaming Huang, Jiaxin Deng, Jinghao Zhang, Junda She, Kuo Cai, Lejian Ren, Lu Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rongzhou Zhang, Ruiming Tang, Shiyao Wang, Wuchao Li, Xiangyu Wu, Xinchen Luo, Xingmei Wang, Yifei Hu, Yunfan Wu, Zhanyu Liu, Zhiyang Zhang, Zixing Zhang, Bo Chen, Bin Wen, Chaoyi Ma, Chengru Song, Chenglong Chu, Defu Lian, Fan Yang, Feng Jiang, Hongtao Cheng, Huanjie Wang, Kun Gai, Pengfei Zheng, Qiang Wang, Rui Huang, Siyang Mao, Tingting Gao, Wei Yuan, Yan Wang, Yang Zhou, Yi Su, Zexuan Cheng, Zhixin Ling, and Ziming Li. 2025. OpenOneRec Technical Report. arXiv:2512.24762. arXiv:2512.24762 https://arxiv.org/abs/2512.24762

[226] Guorui Zhou, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Shiyao Wang, Weifeng Ding, Wuchao Li, Xinchen Luo, Xingmei Wang, Zexuan Cheng, Zixing Zhang, Bin Zhang, Boxuan Wang, Chaoyi Ma, Chengru Song, Chenhui Wang, Di Wang, Dongxue Meng, Fan Yang, Fangyu Zhang, Feng Jiang, Fuxing Zhang, Gang Wang, Guowang Zhang, Han Li, Hengrui Hu, Hezheng Lin, Hongtao Cheng, Hongyang Cao, Huanjie Wang, Jiaming Huang, Jiapeng Chen, Jiaqiang Liu, Jinghui Jia, Kun Gai, Lantao Hu, Liang Zeng, Liao Yu, Qiang Wang, Qidong Zhou, Shengzhe Wang, Shihui He, Shuang Yang, Shujie Yang, Sui Huang, Tao Wu, Tiantian He, Tingting Gao, Wei Yuan, Xiao Liang, Xiaoxiao Xu, Xugang Liu, Yan Wang, Yi Wang, Yiwu Liu, Yue Song, Yufei Zhang, Yunfan Wu, Yunfeng Zhao, and Zhanyu Liu. 2025. OneRec Technical Report. arXiv:2506.13695. arXiv:2506.13695 https://arxiv.org/abs/2506.13695

[227] Yuhang Zhou, Zhuokai Zhao, Ke Li, Spilios Evmorfos, Gökalp Demirci, Mingyi Wang, Qiao Liu, Qifei Wang, Serena Li, Weiwei Li, Tingting Wang, Mingze Gao, Gedi Zhou, Abhishek Kumar, Xiangjun Fan, Lizhu Zhang, and Jiayi Liu. 2026. LLM-Driven Reasoning for Constraint-Aware Feature Selection in Industrial Systems. arXiv:2603.24979. arXiv:2603.24979 https://arxiv.org/abs/2603.24979

[228] Jieming Zhu, Quanyu Dai, Liangcai Su, Rong Ma, Jinyang Liu, Guohao Cai, Xi Xiao, and Rui Zhang. 2022. BARS: Towards Open Benchmarking for Recommender Systems. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 2912–2923. arXiv:2205.09626 doi:10.1145/3477495.3531723

[229] Jieming Zhu, Jinyang Liu, Shuai Yang, Qi Zhang, and Xiuqiang He. 2021. Open Benchmarking for Click-Through Rate Prediction. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. ACM, New York, NY, USA, 2759–2769. arXiv:2009.05794 doi:10.1145/3459637.3482486

[230] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. 2025. Establishing Best Practices in Building Rigorous Agentic Benchmarks. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*. arXiv:2507.02825 https://openreview.net/forum?id=E58HNCqoaA

[231] Zijie Zhuang, Changxin Lao, Pengbo Xu, Hanwen Xu, Ruochen Yang, Yingzhi He, Peng Zhang, Jiangxia Cao, Yusheng Huang, Guohong Mu, Jian Liang, Ruiming Tang, Shuang Yang, Zhaojie Liu, Wenwu Ou, and Kun Gai. 2026. From Trajectories to Evidence: Auditable Experimental Records for Industrial Research Agents. arXiv:2608.05235. arXiv:2608.05235 https://arxiv.org/abs/2608.05235

[232] Zhuang Zhuang, Zhipeng Wei, Rongfeng Guo, Shijie Li, Peng Zhao, Jie Chen, and Fei Pan. 2026. EvoReason: Self-Evolving Reasoning Primitive-Guided On-Policy Distillation for Latent Reasoning in Generative Recommendation. arXiv:2607.29010. arXiv:2607.29010 https://arxiv.org/abs/2607.29010

A The Minimum Tier Applied to Two Papers

Table 12 applies the minimum tier of Table 9 to an open recommender harness (RecHarness) and to the comparison point that meets the most criteria (EvoRank), using only the audit evidence. It illustrates, on two papers and with one rater, that the checklist can be checked from a paper's text, and that the LLM calls needed to meet most of it can be cheap (about ten dollars per run in EvoRank, a learning-to-rank comparison point rather than a recommender loop; we have no recommender-loop figure on the same basis); training the candidates, which this figure leaves out, usually costs more.

Notes on the retrospective proxy mapping (Table 10). R1 reads RecEvolve's candidate set as sampled, as F1 does, although it is stated only in its incident description; the stricter reading gives one fewer. The R1, R2, R6 and R8 proxies are lenient: an AUC loop counts without stating the traffic it is computed on; an online-selecting loop counts by its online gate, without checking that its proxy and offline stages are named; any stated budget counts rather than a full budget card; and any stated access design counts, without checking that every incident is reported, even where it shows that the agent could reach the test data, as for RankEvolve-HSTU, so R8's count is one above the 23 loops with an isolation design in Table 6. For R7, the three papers coded as using untuned defaults may also meet the minimum if they say so; most loops compare against their production model, whose tuning is rarely described, and one of the three counted (Population Evolution) calibrates its shared baseline configuration on the test split. For R5, the proxy counts any equal-budget non-LLM search; EvoSkillRec's NASRec searches a different space and NOVA's null is on one task with unmatched LLM cost, so under the recommended definition, a non-LLM null over the loop's own edit space at matched compute, LLM

cost included (HPO over the changed knobs for a configuration-level loop, random code mutation for a code-level one), the count is one, VirtualMLE (the second R5 row, a judgement rather than a coded cell), a configuration-level loop compared with grid search and Bayesian optimisation (F6). For R9 (min.), two of the four loops counted (NOVA, EvoPilot) assert significance without an interval or a named test; counting only reported intervals or tests (EvoRec's two-sided test, Astar's $p < 0.05$), the figure is 2 of 19 (of the seven loops F9 counts with significance or an interval, three report an interval or a named test, Appendix C; the third, Google, states no duration).

Table 12: Minimum-tier compliance of two audited papers (a core loop and a comparison point), from the released evidence; each item is scored on what the paper reports, not on the quality of the practice. Y met, P partly met, N not met, – not applicable.

	RecHarness	EvoRank	Note
R1	Y	Y	99 sampled negatives stated; per-query full lists
R2	Y	Y	both select on a validation split
R3	P	Y	RecHarness tests once but reports no validation score beside it
R4	P	Y	three runs in both (RecHarness: on its Amazon datasets, not stated for KuaiRec), RecHarness reports only their mean
R5	Y	Y	both state their nulls and budgets (RecHarness's random arm router still lets the LLM write the edits, a weak null, but the minimum asks only for the statement)
R6	P	P	RecHarness gives GPU time and trials but no LLM tokens or cost; EvoRank gives iterations, dollars and time per evaluation but no GPU-hours or human hours
R7	N	Y	cold-start template vs. Optuna-tuned LambdaMART
R8	P	Y	isolated workspaces vs. leakage-guarded helpers and a pre-search audit
R9	P	–	7-day A/B on 10% of traffic; statistical significance not stated

B Coding Provenance and Reliability Details

This appendix gives in full the coding history summarised in §9.

Search counts and bibliography checks. Only hits dated 2025 or later were screened. Summed over the 190 queries (a hit returned by two queries counts twice), the queries returned 1,609 hits; 1,170 of them, dated 2025 or later, were read, and 1,000 of those were not already in the corpus and were screened by title (817 distinct records, the count in Table 1). The other 439 hits were not read: 102 were older than 2025, and 337 lay beyond the 60-hit read window of four broad queries and were not dated, so 1,170 is a lower bound on the 2025–2026 hits.[8] The concentration of the corpus in 2025–2026 is therefore partly a result of the search, and the second sweep cannot have found earlier systems. Every arXiv entry in our bibliography was generated by script from the arXiv API record. The separate check of entries, citations and code repositories (`review/ref-check.md`) was run on the bibliography of 22 October. On 29 October we re-ran the citation check and the arXiv title, author and date comparison over every entry, and each remaining difference is a published version's title, author list or year, or accent formatting. Code repositories were not re-checked.[9]

Systems not audited. Meta's REA is described only in a blog post and is not audited, so 33 of the 34 systems of the abstract are in scope for the audit. Two NeurIPS 2026 workshop papers are known to us only from their abstracts: MLE Agents, which kept 325 of 1,097 agent-written changes to production search rankers [91] (its title places it in recommendation, so we count it as in scope provisionally, unlike the search-ranking comparison points; if the full text confirms search ranking only, it moves to the comparison points and the 34 becomes 33), and an open-ended research loop on the public KuaiRand [45] and KuaiRec [44] data [27]; we will code them once the full texts are available. AgenticTagger [185], RuleAgent [176], Agent0 [158] and AutoModel (a case study) [208] were recognised as in scope only after the second coder's sheets had been returned; we report the counts without them rather than add rows coded by one reader to a table coded by two, and will code them by the same procedure in the next version of the released table. Four borderline systems were read but not counted: RecSys Factory [4] (an operations platform in which an operator's approval, not an evaluation, keeps or rejects each proposed override), AutoRecLab [8] (experiment generation), AURA [77] (stops at pull requests) and Spotify's prompt loop [134] (an L4 system). Feedback-grounded policy discovery [29] evolves per-user LLM-teacher text, an L4-like object outside the audit's scope.

The first coding was drafted with LLM assistance: LLM agents (Claude) extracted a verbatim quote for each cell from the full text, and the first author coded the cell from that quote and its context, checked against the paper text but without a full reading of every paper. Two adversarial verification passes, run by LLM agents and reviewed by the authors, then checked the cells of the rows found in the first sweep against the original papers: the first corrected 12 cells in nine rows, and the second re-verified every sentence that carried a citation at the time against its source (the full text for audited systems and for sentences with a number or quote, the abstract for the remaining L4 and background citations). The seven rows added by the second sweep were checked afterwards by a further agent pass against the full text (147 cells and the sentences citing them; released as `audit/verification-pass-3.md`), which changed one comparison-point cell and two sentences. The first pass's fact check and the last pass's report are released with the audit data, the second pass's report in `review/ref-check.md`; the first pass predates the changelog. A later recoding, prompted by an internal review, recoded nine criterion cells and five object labels,

[8] All three sums are per-query and come from the released query log. The four broad queries (for example `harness` with `recommend*`, 265 hits; 577 in all) were read only to their 60 most recent hits, all dated 2025 or later; every other query was read to its last hit (the largest has 65). 52 queries were also limited by submission date in the query itself (41 to 2026, ten to June 2026 or later, one to 2025 onwards; each window is in the log).

[9] These bibliography checks and the matching step of the recall check read the bibliography and manuscript source, which the supplement does not include, so they cannot be rerun from it.

moved meta-evolution into its own column, and separated C9 into isolation design and reported incidents. Every later change to a cell is listed, by data version, in the released cell-level changelog; where an evidence file and the data file differ, the data file is authoritative, and each system's section in the evidence files opens with its released codes and the cells recoded since first coding, and each verdict line of a recoded cell carries its released code. A generated file, `evidence-released.md`, gives one row per cell with only the released code and the evidence that supports it now; each of the 82 credited cells has a quote and pointer there. For 21 of them the first-coding note was a paraphrase, and the verbatim quote was added in a later pass and matched against the coded arXiv version (`quotes-added.json`).

Another author then coded all 27 core systems from the full texts with the same codebook, with the coding guide's instruction not to open the first coding's files before returning the sheet (the guide did not ask the coder to avoid LLMs). By the second coder's own account, the papers were read by hand without an LLM, and neither an earlier version of the survey's audit table nor the draft's findings had been seen before the sheets were returned; whether the coder opened any other first-coding file (the sheet, the quote files or the evidence files, all in the shared repository) was not asked and is not recorded, so compliance with the guide's instruction not to open them is not confirmed, and 17 first-coding cells were visible in the codebook file the guide pointed to. The coding guide used by the second coder was written after the first coding by an author who knew the corpus: some partial-value definitions describe situations met in the corpus (for C4, a rule to re-run noisy candidates or a noise estimate of single models; for C6, a "random" ablation in which the LLM still proposes, or replay only), and its one quote-format example comes from a corpus paper, which may have aligned the two codings. Two of the guide's definitions are narrower than the codebook's: its incident flag asks whether "the loop itself" gamed the metric, and its C6 question asks for a search "over the same space". The released codes follow the codebook (incidents whoever introduced them; any equal-budget non-LLM search, with the same-space count given separately in F6); both coders nonetheless coded EvoPilot's incident and EvoSkillRec's C6 as yes, and the second coder's other C6 codes are the released ones in all but one loop. The adjudication of disagreements used the guide, but no resolution turned on either narrower definition, and re-reading the resolved cells under the codebook changes none (noted in `reconciliation.md`). Before reconciliation, the two codings agreed on 84% of the 243 cells (criterion-stratified $\kappa = 0.71$, i.e. Cohen's κ with chance agreement taken within each criterion and then pooled, which summarises the per-criterion values that are our primary statistic (Table 7); pooled over all labels 0.81, Gwet's AC1 = 0.84; the first coder's pre-reconciliation sheet is released so that these values can be recomputed). That sheet is the data file as committed on the evening of 22 October, before the second coder's sheets entered the repository on 23 October. The SHA-256 checksums of that sheet and of the two returned sheets, with the commits they match, are listed in `audit/second-coder/README.md`, so the released files can be checked against the repository record when it is opened after review. The second coder received ten systems that afternoon and the other 17 with that commit. Between the two, an internal review changed four scored cells of the first coding (Population Evolution C2 and C8, from the first batch; EvoPilot C2 and C3, from the second) and split C9 into isolation and incident, and the codebook file that the second coder was pointed to listed these and other C9 changes with their new values, and named the four rows that now carry an incident (17 cells in all, mostly C9 and incident). Against the first coding as it stood when each batch was sent (Population Evolution's two cells reverted), pooled κ is again 0.81, and leaving out the 17 cells whose values the codebook file showed also gives 0.81 (0.69 within criteria, and an unweighted mean of per-criterion κ of 0.71 over the eight criteria whose κ can still be estimated; both released). Without those cells the incident flag's κ cannot be estimated: all four reported incidents were among them, so the remaining cells hold no agreed incident and one disagreement (AutoLR), and the 0.87 below is not evidence of independent agreement on incidents; the count of four rests on the papers' own explicit reports. Independence rested on agreement, not on access control: the first coding's files were in the shared repository, and the guide asked the second coder not to open them or the survey's audit table or figure; the second coder had not read the draft's findings either, and the coding time was not recorded (the guide suggested about 20–30 minutes per paper). By the repository record, the ten-paper batch was sent at about 15:00 and the 17-paper batch at about 20:40 on 22 October, and both returned sheets were committed to the repository at 11:04 on 23 October, about 20 hours for all 27 papers; a turnaround of under 45 minutes a paper, overnight included, for full texts with long appendices limits how deep and how independent a check the second coding can be, and the second coder had access to the shared repository, including the first coding's files, throughout; the second coder reports using no LLM, and we have no record of how long each paper took or how much of each appendix was read, so the agreement figures describe this coding and not a slower one. By criterion, κ was 0.67 for C1 (95% bootstrap interval 0.46–0.86), 0.90 for C2, 0.79 for C3, 0.91 for C4, 0.30 for C5 (−0.04–0.63), 0.65 for C6, 0.57 for C8, 0.77 for C9 and 0.87 for reported incidents (visible; see above); AC1 was at least 0.70 everywhere except C5 (0.50). The unweighted mean across criteria is 0.71, and excluding the 11 cells that both coders marked not applicable leaves the pooled value at 0.80. The pooled value treats all labels as one set; because the criteria use different labels, this lowers the chance term and raises κ. Taking chance agreement within each criterion instead (cell-weighted) gives 0.71 (95% bootstrap interval over systems 0.60–0.79), again 0.71 against the first coding as each batch was sent, and 0.69 without the 17 visible cells; we report both.

The 38 disagreements concentrate in C5 (10), C1 (7), C8 (6) and C6 (5). In C5 the second coding was more favourable in six and stricter in four; in C6 it was stricter in all five (partial controls the first coding had credited); in C1 and C8 most disagreements were about which label applies (for example AUC versus online-only for loops whose headline is an online lift), not about whether a practice was followed. They were resolved against the paper text, with LLM agents drafting a resolution and quote for each and both authors reviewing them; the released coding sides with the second coding in 31 and with the first in seven (counted after the EvoSkillRec override below, which sides with the second coder; the drafts alone gave 30 and eight), so the first coding was changed in 31

Table 13: Headline counts of the scored criteria under each coding: the first coder's sheet before reconciliation, the second coder's returned sheets, and the released coding. F1–F6 are over each coding's own offline-selecting loops. The first coding predates the rules written during reconciliation (for example, that an online test of an offline-selected candidate is partial transfer evidence, which accounts for most of the change in F5), so its column differs partly by rule, not only by reading. "+1" in F2: one loop the first coding coded as selecting on training loss. Computed by `summarize.py` from the released sheets.

Count	First coder	Second coder	Released
Offline-selecting loops	17	16	16
F1 ranking: full / sampled / not stated	3 / 2 / 5	3 / 3 / 4	3 / 3 / 4
F1 AUC	6	3	5
F2 selection: validation / reported data / not stated	5+1 / 4 / 7	4 / 4 / 8	4 / 4 / 8
F3 test-once: yes / not stated	3 / 10	2 / 8	2 / 8
F4 spread over sessions: yes	0	0	0
F5 held-out check: yes / partial	2 / 8	1 / 14	1 / 14
F6 null: yes / partial / none	3 / 9 / 5	3 / 4 / 9	3 / 5 / 8
F7 incidents (all 27)	4	5	4

of 243 cells (13%), all of them errors if the LLM-drafted, author-reviewed resolutions are right. Counting from the second coder's sheet alone (Table 13, which gives each headline count under both codings and the released one) gives the same 16 offline-selecting loops and the same C2–C5 counts as the released coding, and C6 differs in one loop (Population Evolution, partial versus no), so the offline C2–C6 counts do not rest on reconciliation. The incident flag and isolation do: the second coder flagged five incidents (AutoLR as well, resolved to no) and coded RankEvolve-HSTU's isolation partial rather than no, so the count of four incidents is the reconciled one. Reconciliation moved Google's loop to the online-selecting stratum (17 to 16 offline-selecting loops) and lowered three headline counts: loops naming a validation split (five to four), loops stating test-once (three to two) and loops with an explicit held-out check (two to one); it also moved partial and not-stated tallies, and with them some ranges: F1's upper bound (eight to seven), loops not stating their selection data (seven, plus one coded as training loss, to eight), F3's range (3–13 to 2–10) and F6's partial null searches (nine to five, with loops without one rising from five to eight). No other yes-count in F1–F9 changed. The protocol was not pre-registered. The coding guide changed between the second coder's two batches: the ten papers of the first batch were coded under a version whose C2, C4, C8 and C9 definitions were later refined and which had no incident column, and the released diff (`second-coder/guide-batch1-to-batch2.diff`) shows each change. The codebook also changed during the work (C9 split into isolation and incidents, meta-evolution moved to its own column, and the post hoc reconciliation rules), each change listed with the data.

Most disagreements came from rules the codebook left open: which gate counts as the selection in a multi-stage loop, whether an online test of an offline-selected candidate is transfer evidence (the guide's partial examples, another dataset, unseen users or a later period, do not name it, and the first coding had no guide), and whether computing test scores that are not used for selection breaks test-once. The rules that settle them were written during reconciliation, after both codings were complete, and are therefore post hoc; they are listed with the data. The online-confirmation rule writes down the second coder's reading, whose C5 codes for the 16 offline-selecting loops it reproduces exactly (the first coding had coded six of the confirmations as no); without the rule F5's partial count falls from fourteen to five, and F5's conclusion, which rests on the one same-task check, is unchanged. One adjudication was overridden by the authors: the drafted resolution kept EvoSkillRec's C3 as "yes" because its test scores are not used for selection, and we coded it "no" under the literal codebook wording (test data used during the search or repeatedly), because the test score is computed for every candidate in every generation ("Selection uses validation AUC only; test scores are reported but never used in the search", App. E.2; Tables 27–28 give validation next to test per generation), as the second coder had coded it; under the other reading, F3's count would be three rather than two. Two criteria fall below $\kappa = 0.6$ over all 27 loops, so the counts that rest on them are the least certain: C5 (F5) and C8 (F8); over the 16 offline-selecting loops C6 (F6) is also below, at 0.54. The design fields (§5.1) the kept-edit tallies, the F8 lift count and the seven comparison points were coded by one author and not second-coded (the strata follow from C2, coded by both authors, and rest on the second coder's reading of one cell: the first coding placed Google/YouTube's selection offline, which would give 17 offline-selecting loops; the comparison points carry part of F5's contrasting evidence and Table 3's proposer rows rest on the design fields), so they are as uncertain as C5 and C8 (for scale: the first coding of the scored criteria was changed in reconciliation in 31 of 243 cells, 13%, and single-coded fields may carry errors at a similar rate), and no human coder outside the author team was involved. The online-gate items of F9 were coded by one author and then re-coded blind by a separate LLM agent (Claude, the model family that extracted the first coding's evidence, so their errors may be correlated) that had no access to the first coding; this is a consistency check, not an independent human coding. The free-text count of rounds is not compared. Before adjudication the two agreed on 88 of 95 cells ($\kappa = 0.86$ stratified by item, our convention elsewhere; per item 0.90–1.00, except confirmation at 0.65; pooled over items 0.91, which inflates κ as noted above), or 79 of 86 without the nine correction cells that are not applicable to confirmation-only loops (of the 19 loops, role 18, duration 18, interval 19 and confirmation 15, or 6 of the 10 online-selecting loops; correction 9 of the 10 loops to which it applies; the first coding before adjudication and the script are released); the disagreements were N versus P, select versus both, one N versus Y (PILOT's correction, below) and one Y versus not stated (DMRL's duration, which we then changed to not stated because its seven days are a metric horizon, not an experiment length); after a later review we also changed AgentX's duration, stated in both codings, to not stated, because only its three-week deployment is given, not the length of each experiment. Under either coding one online-selecting loop confirms its final candidate in a fresh experiment. The one correction disagreement is PILOT: our first coding was "no", the blind coding found and credited its joint calibration of parallel challengers within a round, and we revised our code to partial because the calibration does not extend across rounds; under the blind coding, one of the ten

online-selecting loops corrects for multiplicity (within a round), and none across rounds under either coding.

Cells resting on reconciliation. The second coder's sheet alone gives the same strata, the same F1 split of ranking loops, the same F2–F5 counts and F6's three loops (one loop moves between partial and none); what rests on reconciliation is seven cells, those in which the released codes differ from the second coder's sheet: NOVA's and Trajectories-to-Evidence's C1 (AUC; the second coder coded other and online, so F1's AUC count would be three, not five), Population Evolution's C6 (partial; none) and C8 (partial; tuned), Self-EvolveRec's C8 (untuned; seed), RankEvolve-HSTU's C9 (no; partial), on which F7's statement that one loop's agents could reach the test position rests, and AutoLR's incident flag (no; yes, which would make five incidents); and one drafted resolution was overridden (EvoSkillRec C3; under the drafted reading F3's count would be three). Two online duration cells were changed to not stated after the LLM re-coding, DMRL's at adjudication and AgentX's after a later review.

Recall check. As a check on recall, run with an LLM agent on 27 October, after the cut-off, and not second-checked, we read the related-work section of each of the 27 core papers and extracted every 2025–2026 cited system that could meet the inclusion criteria: 22 of 24 were named in the survey when the check was run, 17 of them coded (16 core and one comparison point) and five named as borderline, adjacent or uncoded, and the two then absent, cited here since, are borderline at most—MoFA, a one-shot LLM feature selection for production rankers without an automated keep-or-reject loop [227], and an LLM orchestrator that optimises recommendation lists rather than the recommender. This check can only find systems that the core set cites, so it bounds recall within the citation network, not outside it, and is not independent of the search, which followed the same citations; that the first sweep missed five of the 27 core loops, and that four more systems were recognised too late for the coding, suggests the corpus is not saturated. Two of those four are from 2025 (RuleAgent, March; Agent0, July), earlier than the 2026 systems the citation walk started from, so earlier work in particular may be under-covered. The first sweep found both, and AgenticTagger, before the cut-off but catalogued them as adjacent feature-engineering, data-rule and generative-component work; they were recognised as loops only after the coding (AutoModel was already cited). The second sweep counted only hits not already in the corpus, so it did not re-examine them: they show a classification miss rather than a search miss. Six known systems in scope are not coded, four recognised too late for the coding and two known only from abstracts (§6), besides Meta's REA (a blog post only) and the out-of-scope policy-discovery loop; adding them could move any core count by at most six (Table 7, column 3; combined with the not-stated ranges, the range for test-once among offline-selecting loops would become 2–16 of at most 22 if all six selected offline and stated nothing).

C Details of Findings F5–F9

This appendix gives the evidence and counting rules behind findings F5–F9 of §6, which the main text states in brief.

F5. In EvoSkillRec, five candidates lie on the validation Pareto front, "but only P1 (round 19) retains both improvements on the test set" [92]; P1 has the highest validation AUC of the five (0.92999 against 0.92927–0.92996),[10] so choosing among the front points by validation would have picked it, although the paper states no rule for choosing one.[11] P2 and P3 lose their AUC gain on test, and P4 and P5 have none on validation. This is the only same-task validation-to-test check in the core corpus, and both coders coded it yes. But the front has two objectives and no stated selection rule, and P1's validation gain over the baseline, 5.1×10^{-4} AUC (0.92999 against 0.92948), shrinks on test to about 1.8×10^{-4} AUC from a single run, of the order the paper says a single run cannot separate from run-to-run variation (F4), so the check shows neither that the selected gain is real nor that it is not. Of the other fifteen, fourteen are coded partial and A-MLE no.[12] All nine online confirmations report a gain, which shows that the selected candidate carries over to traffic, though only for the one candidate that reached it. The evidence of gains that do not survive comes from transfer across datasets and scale, which asks whether a gain generalises rather than whether the selection data were overfitted, and from comparison points outside recommendation. Population Evolution checks transfer across datasets and scale: a ranking candidate gains 10.66% on a peer dataset and loses 0.79% at larger scale, two of three peer-screened designs keep their gains, and sharing evidence across searches raises the larger-scale pass rate for ranking from 11/60 to 18/60 and for RL from 0/15 to 2/12 [137]:[13] in ranking and RL, most locally selected candidates do not graduate. Two other loops carry selected recipes or memories to other data, without a larger-scale step: RankEvolve-HSTU replays its recipes on four further datasets, one run each, and in Trajectories-to-Evidence three of five memory transfers help and one hurts [28, 231]. Two further observations suggest that headline values are optimistic. Amazon's headline lift of 1.82× is the peak of a trajectory, "~2.6σ above the across-date mean" of 1.63× [18], and Kuaishou's trajectory analysis finds that for 22 of 30 methods the last round falls below an earlier best [231]. The comparison points give larger and mixed effects: in EvoRank, the gains of the first (loss-evolution) campaign "were almost entirely fitness noise" on 60k held-out queries, while about 70% of the second campaign's gains survived [136]; in an e-commerce search loop, effects on a shadow-traffic proxy were 17 and 29 times larger than in decision-grade online experiments for the two candidates traced, on different metrics per stage, and five promoted candidates showed no significant gain [173]; and in Evolving Semantic Indexes, a proxy decoder that ranks semantic-ID programs well (Spearman $\rho = 0.90$) becomes "not only less informative but misleading" once the training objective may change, on a small sample of five programs [129]. A controlled study shows that the mechanism exists: on a label-permuted task with no real signal, a coding agent given exact validation scores "ended by reporting a gain of 0.0157" [196].

[10] The paper's Table 4, transcribed with all five rows in the released design evidence.
[11] A candidate off the front with higher validation AUC failed an eligibility constraint.
[12] Online confirmations are coded partial by a reconciliation rule; see Appendix B.
[13] For watch-time it rises from 4/10 to 5/7.

F6. The three null searches differ in scope.[14] Some of EvoSkillRec's winning margins are of a size it notes a single run cannot separate from run-to-run variation (F4), and stricter definitions of the null give two or one (Appendix E). The partial controls are NAS baselines at unmatched budgets, a random arm router that still uses LLM edits, a replay-only random policy, or a starting model tuned by an optimiser that the LLM partly steers; comparisons with human workflows or with other LLMs do not count. No loop runs the catalogue-of-known-techniques null of §5.2. The two random policies we found were not dominated, although neither is a null over a whole loop: in AgentX-Model's historical replay, which randomises the choice of experiments rather than the edits, in Scenario A "Uniform Random's mean exceeds Fixed, Greedy, and Joint" policies, and no policy led consistently across scenarios [194]; in EvoRank's loss campaign an equal-budget random search transferred as well or better ($p = 0.03$) [136]. AutoLR states plainly that it "does not establish superiority over random, greedy, or direct LLM-based routing under a matched trial budget" [213]. The history of architecture search explains why the control matters: random search with early stopping matched leading NAS methods [89], the search phase of several methods was no better than random sampling [190, 200], and classical optimisers beat LLM agents in fixed hyperparameter spaces [41, 148]. The online-selecting loops compare each candidate with a control arm, which is a different question from whether the proposer beats a null search.

F7. RecEvolve's agent, "in a separate exploratory session" of the same system, "discovered that drastically reducing the batch size (from 8k down to 1k) artificially inflated proxy metrics"; a human caught it and rolled it back [135]. On HSTU, RankEvolve-HSTU's agents produced a genre feature that leaked the target,[15] DPO preference pairs mined at the held-out target position, and a naive multi-position evaluation that would have inflated NDCG@10 from 0.2192 to 0.2517/0.2577; the genre and DPO bugs surfaced as anomalous trajectories on the leaderboard, and the inflation was reasoned out as leakage in the diagnosis the loop requires for every negative result, but the best agent pair still had a 10.4% silent critical-defect rate on ExecML-HSTU, and a single Claude Code pass 35.4% [28]. In EvoPilot, "a favorable result used an evaluation date inside the candidate model's training window, so the result was retracted" [42]; the paper does not say whether the agent or a person chose the date, or what caught it. In Amazon's semantic-ID loop, "the agent discovered that maximizing codebook size trivially improves weighted coherence", and the resulting hierarchy was unusable; human QA between runs caught it [18]. Preventive guards leave no incident to report: EvoRank audits its data for leakage before the search starts [136], and NOVA's verifier rejects leaking edits before they are measured [109]. Coding agents that can reach their tests behave the same way outside recommendation [6, 122, 224]. Among the borderline systems, AURA's agent fabricated 904 session identifiers in its diagnostic stage [77], and RecSys Factory's diagnostics uncovered a pre-existing upstream join bug that had "silently corrupted all downstream training samples" [4], a reminder that the data feeding the loop need checks too. As designed, only VirtualMLE states both that the test set is hidden from the agent and that the data pipeline is immutable (AgentX-Model states that its test split and acceptance rules are outside its repair agent's control, with fixed evaluation code, and is counted among the partial loops); eleven offline-selecting loops constrain part of the loop (verification gates, fixed evaluation code, whitelists, isolated workspaces, sandboxed modules), RankEvolve-HSTU lets its agents run evaluations at the test position, and three say nothing; all ten online-selecting loops bound the agent's actions.

F8. Two examples of weak starting points:[16] RecHarness improves average HR@10 by 85.85% over a cold-start GRU4Rec [58] template and by 12.58% over HSTU [102], and Self-EvolveRec configures all models with the same fixed hyperparameters (embedding 50, batch 128, learning rate 0.001) without describing tuning them [78]. Against production baselines, we counted as broad the metrics a whole surface is judged by (watch time, engagement, orders, revenue or GMV; listed per loop in the released `f8-lifts.md`); the 17 reported lifts on such a metric range from DMRL's +0.052% LTV to Astar's +4.86% GMV.[17] The range is descriptive: its metrics, designs and baselines differ (some lifts come from a selecting round and PILOT's is not randomised), so it does not support comparing loops. Small lifts are normal in production and are not by themselves a sign of weak evaluation. Larger figures are on narrower metrics, such as AgentX-Model's 10–15% acquisition efficiency or Trajectories-to-Evidence's +6.34% user-growth net utility, or come from a selecting round rather than a confirmation, such as Sortify's +9.2% GMV in its first market.[18] Only VirtualMLE and Amazon's AutoResearch (a hand-tuned baseline from four engineer-weeks) describe tuning their baselines, and only VirtualMLE says on which data ("all tuning decisions are made exclusively on the validation set"); Population Evolution tunes them on the test split (C8 coding: Table 6), which is why the lenient R7 proxy in Table 10 counts three. Budgets are reported in incompatible units (iterations, GPU-seconds, GPU-hours, ideas, tokens, dollars, yuan).

F9. The online counts are coded by one author.[19] Online experiments play two roles.[20] In nine loops that select offline, the A/B test checks one candidate chosen beforehand, so it is a confirmation by construction.[21] In the ten online-selecting loops the experiment is the keep-or-reject rule, and there the headline effect is usually the selected round: A/B Agent's headline GMV lift repeats the numbers

[14] NOVA's Optuna-TPE runs at a matched trial budget but unmatched LLM cost, and only on its ScaleUp task, a search over structural hyperparameters. EvoSkillRec's NASRec trains 200–250 models against the loop's 200 candidate slots, single seed.

[15] Training loss collapsed to zero while evaluation spiked.

[16] We did not compare gain with baseline type systematically.

[17] Exclusions and alternative endpoints are in Appendix E. Six of the 17 list several values or an 'up to' figure, all inside this range. Without PILOT's non-randomised before/after lift the range is unchanged. AutoLR reports only lifts summed over nine launches and is not counted. GEARS reports per-surface lifts on anonymised metrics ("Metric 1", +0.011% to +0.14%), so whether they are broad cannot be told; counted, they would lower the floor to +0.011%.

[18] For these two we count Trajectories-to-Evidence's +0.75% page duration and Sortify's confirmed +4.15%. The rule is to count a fresh confirmation where one is reported and otherwise the paper's reported figure, which for A/B Agent (+4.829%) is the selecting round itself (F9).

[19] The partial codes are also unstable: the blind LLM re-coding gives the same totals but agrees on only two of the four partial loops (GEARS and PILOT), placing the other two on Google's loop and AgenticRecTune instead of SR-Agent and DMRL (Appendix B).

[20] See columns `ab_*` of the data file, with quoted evidence.

[21] For AgentX-Model and AutoLR, however, it is the latest or the passing launch reviews that are reported, itself a selection.

of its fifth strategy round [68], and Sortify's abstract reports an in-loop round. Across all 19, eight state the experiment duration and seven claim significance or report confidence intervals for the headline result, three of them with an interval or a named test.[22] No online-selecting loop corrects across its rounds,[23] although the arm chosen as best overstates its effect [11, 85]; AutoLR discusses the problem, and its summed launch-review lifts are, in its own words, "descriptive and are not pooled treatment effects" [213]. Only Sortify re-tests its chosen round in a fresh seven-day experiment [30]: "the frozen R7 parameters were validated through a 7-day A/B test" in a second market, so the candidate is the one selected, although the change of market also makes it a test of transfer. Four do so in part: PILOT designs its final confirmation as "a single pre-registered test" that "runs once", because "the data collected while searching … cannot also serve as proof" [97], but the report does not yet include that confirmation, and its headline compares the best of five buckets before and after, "not randomized"; SR-Agent's system-level experiment runs alongside its in-loop experiments; GEARS asserts a significant lift for its case-study deployment without giving numbers; and DMRL does not say whether its comparison windows are separate from its training rollouts. Online noise is visible inside single papers: DMRL's full configuration appears in six tables, said to run in "non-overlapping time windows", with LTV lifts from +0.021% to +0.064%,[24] a spread of 0.043 percentage points, larger than its 0.028-point lead over the best baseline in the main table; the authors restrict their conclusions to within-table comparisons [216].

D Catalogue of Systems

D.1 Configuration-level loops (L1)

L1 systems let an LLM change numbers or switches while the model code stays fixed. VirtualMLE tunes SASRec and HSTU over a predefined action space of hyperparameters and pre-implemented structural switches (so its kept architecture options, such as relative position bias or a SwiGLU feed-forward block, are switch settings, not code the agent writes) through an execute–reflect loop with a hierarchical memory that transfers across datasets [16]. In production, CORAL allocates retrieval budgets and serving treatments [5], AgenticRecTune tunes fusion weights and thresholds across pre-ranking, ranking and re-ranking [184], Sortify searches the exchange rates of a sorting formula with Optuna [2] while an LLM meta-controller recalibrates the search's offline-to-online transfer intercepts and penalties [30]. GEARS selects segment-level treatment policies [203], and SR-Agent edits post-ranking diversity and exposure strategies through four bounded action types [192]. Taobao's PILOT has LLM roles choose ranking-objective weights for user segments from a legal-command envelope generated by deterministic services [97]; Kuaishou's A/B Agent iterates strategy parameters across online rounds with a hierarchical experience tree [68]; and DMRL trains a small LLM with RL to rewrite the skill document that an ad-tuning agent follows [216]. LLM-driven hyperparameter optimisation [110, 212] is the general-ML form of L1 and is routinely compared against Bayesian optimisation or random search, the null control that few recommender loops run (F6). Most L1 systems in production select directly on online experiments; an e-commerce search loop adds a multi-fidelity ladder from shadow-traffic proxies to decision-grade online tests [173].

D.2 Code-level loops in the open (L2)

Self-EvolveRec [78] runs an AlphaEvolve-style loop over the whole pipeline code of NCF [55], NGCF [172], SASRec and MoRec [201] seeds; besides validation metrics it receives directional critiques from an LLM user simulator and from a diagnosis tool that evolves with the model. **RecHarness** [102] routes each round with Thompson sampling over human-defined "edit arms" (schedules, dropout, depth, pooling, features, loss) plus "jump" arms for structural changes, uses validation feedback only, evaluates the incumbent on test once, and caps compute at 43,200 GPU-seconds; HSTU is one of its five backbones. **EvoSkillRec** [92] evolves architectures as typed "skill genomes" of executable modules and promotes validated modules into a skill library. **RankEvolve-HSTU** [28] composes Claude Code and Codex under an executable operating protocol in which the two agents review and repair each other's edits to HSTU, and introduces ExecML-HSTU, a benchmark of how often agent edits are executed correctly. **Population Evolution** [137] runs several evolution searches that share evidence across related datasets, screens candidates on peer datasets (and, for LLM and VLM targets, on small training subsets), and promotes survivors to larger scales; it introduces RMD-Bench. AutoRecLab [8, 9] generates recommender experiments from natural-language descriptions; it automates experimentation rather than evolving a model, and we do not audit it. Two loops evolve data and features rather than the model: RuleAgent has a language agent propose rules for denoising implicit feedback, retrains with an unlearning shortcut, and keeps rules by validation performance [176], and Agent0 tunes the prompts that turn raw text into categorical features through feedback from a downstream oracle [158].

D.3 Code-level loops in industry (L2)

Most industrial systems share a two-stage design: an offline loop screens agent-written candidates, and online experiments decide launches. Google's loop for YouTube ranking [170] has Gemini personas propose optimiser, architecture and reward changes, promotes on validation loss (with a proxy metric for reward changes) and then on north-star A/B metrics, and reports LLM costs of about $20,000 over six months. RecEvolve [135] runs the full research loop on a production two-tower retrieval model. Meta's REA plans, launches and debugs ads-ranking experiments with up-front GPU budget approval [121]; A-MLE explores a portfolio of ads-ranking models with human checkpoints [46]; Auto-RecSys runs idea- and execution-evolution loops over multi-day training jobs [90]; and RankEvolve-HSTU and Population Evolution above also come from Meta. Alibaba's EvoRec [123] pairs research and code agents with a skill evolver, and Astar [60] trains an 8B model on the commit history of an advertising system to propose the next evolution direction. From Astar's authors, AutoModel organises an industrial

[22] Counting explicit claims of significance, which NOVA, CORAL, AgenticRecTune and EvoPilot make without showing a test; counting only reported intervals or tests gives three: Google, EvoRec and Astar. RecHarness's "significant gains" is not counted, because it does not say the gains are statistically significant; counting it would give eight.

[23] PILOT calibrates its thresholds jointly across the challengers tested in parallel within a round.

[24] Five distinct values; one table repeats the main table's numbers exactly.

recommender as evolution agents for models, features and online experimentation, with a case study of reproducing published methods and comparing them offline against production [208]. AgentX, deployed at Kuaishou [84], iterates online strategies and model research, and its successor AgentX-Model [194] runs research lineages (reproduce, follow up, compose, diagnose) over production rankers. A further Kuaishou paper converts research-agent trajectories into auditable evidence [231]. Tencent's NOVA [109] adds verification gates that reject recommender-invalid edits such as target leakage, NetEase's AutoLR [213] reviews proposals with a multi-expert proposal council under a limited trial budget, and Tencent's RecSys Factory bounds agent autonomy to typed decision points in fixed pipelines [4]. A Trip.com co-scientist for travel search ranking [182], LLM evolution of learning-to-rank pipelines (EvoRank [136]) and of lexical retrieval functions [130] are close relatives outside recommendation. Meta's EvoPilot runs a 37-day campaign of agent-proposed configuration and retraining changes to video-retrieval models behind a fail-closed verifier and catalogues the failures it met [42], and Amazon's AutoResearch rewrites the training code of a book-retrieval model and tunes a semantic-ID tokenizer in a twelve-week campaign of more than 220 experiments, naming the failure modes it met [18]. Adjacent loops evolve ads kernels [94], off-policy-evaluation code [181], search-relevance models [169] and an LLM policy layer trained against a learned evaluator [207], and an operations agent for Kuaishou's search engine evolves its skills from on-call engineers' corrections [103]. Feature-engineering loops are close relatives: an error-driven loop turns AUC ranking inversions into feedback for an LLM feature generator [7], a planner-guided agent writes feature code for recommendation models with human checkpoints [166], and LATTEArena compares LLM feature-engineering methods on cost as well as accuracy [52].

D.4 Components of generative recommenders (L3)

Generative recommenders expose objects that do not exist in ID-based models. Evolving Semantic Indexes, a generative document-retrieval system that we treat as a comparison point, runs LLM-driven program evolution over semantic-ID construction, collision handling, decoding and the generator's training objective, using a one-million-parameter proxy decoder as fitness [129]. AgenticTagger has an LLM refine a hierarchical item-descriptor vocabulary used as tokens [185], and a generative recommender can reassign its own item tokenisation during training [24]. Amazon's AutoResearch is the only loop in our corpus that edits a production-scale semantic-ID tokenizer, and the only one in which the agent gamed a tokenizer metric [18]. Feedback-grounded policy discovery distils LLM-discovered natural-language policies into a semantic-ID generator [29]. Evolving Semantic Indexes evolves TIGER-style semantic-ID components for document retrieval but releases no code; no open system yet does so for recommendation under a shared protocol.

D.5 Adjacent work: self-improving agentic recommenders (L4)

When the recommender is itself an LLM agent, the evolving object is its memory, skills, prompts or weights, and the feedback is mostly generated by the system. We catalogue this line as adjacent work and do not audit it (§2). Memory and reflection methods update user and item agents (AgentCF [209], AgentCF++ [108]), per-user policy skills (SAGER [165]), propagation modules optimised with textual gradients (RecNet [87]), and structured memories that are rewritten as interactions arrive (AtomRec [62], AMEM4Rec [128], STEAM [95], MARS [154], MemRec [25]). Others build hint bases by reflection (Re2LLM [174]), refine per-user rubrics from user feedback (RecPilot [133]), or let recommendation and user agents refine each other's inferences over rounds without amplifying popularity or position bias [15]. CoVeMem instead replaces rewritten text memory with frozen collaborative-filtering states learned by gradient training [22]. Weight-level self-improvement includes OneRec's alignment on its own samples, first by iterative DPO [34] and then by on-policy RL [225, 226], self-play debiasing (SPRec [43]) and RL against a recommender's metric (Rec-R1 [98]). It also includes co-evolving recommender and user agents (CoARS [175]), reasoning primitives and a teacher updated from a student's on-policy outcomes (EvoReason [232]), and recursive self-training on filtered synthetic sequences [54, 210], where progress can also sit in earlier generations, not only in the latest model [186]. RecPrompt iteratively rewrites a news recommender's prompt with an optimiser [105], and Spotify improves a conversational recommendation agent's prompts, few-shot examples and tool descriptions with a coding agent, evaluated by an LLM judge on synthetic conversations [134]. Simulated environments and benchmarks for such agents include RecoWorld [106], the user simulators Agent4Rec [206] and RecAgent [171], and the verifiable τ-Rec benchmark [125]. A controlled pilot found multi-agent recommendation pipelines not uniformly better than a single-shot baseline [118], a reminder that L4 gains also need controlled baselines.

D.6 Evolving the evolver (meta)

Eleven of the 27 audited loops change their own proposer. AgentX's SGPO applies semantic-gradient updates to its agents' instructions and validation rules [84]. EvoRec, AgenticRecTune and EvoSkillRec grow skill libraries from experiment outcomes [92, 123, 184], Auto-RecSys rewrites execution playbooks [90], and VirtualMLE and PILOT carry tuning rules or methodology across datasets and experiments [16, 97]. Astar retrains its proposer on each iteration's verified outcomes [60]. NOVA turns confirmed semantic errors into verification rules and may commit validated rules to its knowledge base for later tasks [109]. AgentX-Model's agent proposes repairs to its own tools and instructions, deployed only after human approval [194], and DMRL trains a small LLM with RL to rewrite the skill document that an ad-tuning agent follows [216]. These are the recommender counterparts of harness optimisation [86, 111, 187]. Outside recommendation, AIDE^2 keeps changes to its own code that perform best on hidden evaluations and shows that they transfer to four held-out benchmarks [160]; separating selection tasks from held-out tasks is what the meta-evolving recommenders do not yet report. Meta-evolution inherits the selection problem one

level up: the harness is kept or discarded by a score of its own. Optimising against a learned or LLM judge overfits the judge [48], and a pre-registered study of LLM research-ideation harnesses by one of us, outside recommendation and not peer reviewed, reports the same at the harness level: a harness optimised against an LLM judge raised the judge's scores while the share of its hypotheses that held under measurement fell [193].

E Alternative Readings of Individual Cells

The findings in §6 state each count under the released coding. The readings below would change a count; each stays within the range given in Table 7, except where stated.

- *F1, RecEvolve's candidate set.* RecEvolve does not describe its candidate set with its evaluation, but states, in describing its batch-size incident (F7), that "the Two-Tower model utilizes in-batch negative sampling". If this is not taken as a statement of the evaluation protocol, RecEvolve's candidate set is not stated and the upper bound is eight; the released code, F1, Figure 3 and the R1 proxy (Table 10) all take the sampled reading, as the second coder did; the first coder coded it not stated.
- *F1, Amazon's catalogue.* Reading Amazon's ranking as not full gives two to six full-ranking loops.
- *F2, Google and Astar.* Google's loop also ranks candidates by validation loss, but online experiments decide which run and launch, so we count it as online-selecting; Astar's loss-decrease rule labels its training data and defines its success rate, but the paper does not say what decides its production iterations.
- *F2, Population Evolution.* Population Evolution's code is the least certain: an early verification pass read its test-split sentence as baseline calibration only and coded its selection data not stated; under that reading this count is three, the not-stated count nine, and F3's six loops coded no become five (F3's count of two is unchanged). The cell was changed during internal review and was among those visible to the second coder, so the coders' agreement on it is not independent (Appendix B).
- *F2, AgentX-Model.* AgentX-Model reports "the highest measured AUC" over its rounds, so its reported number is its selection statistic and its C3 is no (the evaluation is used repeatedly), but it never names the data that AUC is measured on; C2 asks which data select, and both coders coded it not stated. Amazon, coded as selecting on reported data, names them ("a held-out scalar metric"). Reading AgentX-Model's reported AUC as naming its selection data gives seven not stated and five on reported data, within the stated range, and R2 19 of 26.
- *F3, EvoSkillRec.* We code it "no" under the codebook's literal wording, test data used during the search or repeatedly, as the second coder did; the first coder and the drafted resolution read it as test-once, because the scores are not used, and that reading gives three (Table 7, Appendix B).
- *F3, Self-EvolveRec.* Self-EvolveRec reports a "Peak" iteration, which suggests reporting the best score over iterations; it is coded not stated because the paper does not say which data that score is on. Reading it as no would move one loop from not stated to no and lower F3's upper bound to nine; the count of two is unchanged.
- *F5, VirtualMLE.* VirtualMLE reports validation and test N@10 for its selected configurations but states that the two are "not directly comparable", so we code it partial; counting it would make the count two.
- *F6, the null's search space.* EvoSkillRec's NASRec searches architectures rather than EvoSkillRec's own code-and-skill space; requiring the null to search the loop's own space gives two, and also requiring a total budget, LLM cost included, at least the loop's gives one (VirtualMLE).
- *F7, the incident count.* The incident code counts a shortcut in a candidate's evaluation inside the loop whoever introduced it; counting only those the paper attributes to the agent gives three. RecEvolve's arose in a separate exploratory session rather than in the reported runs; we count it because it arose in the loop's own evaluation of a candidate, and without it the count is three (two attributed to the agent).
- *F8, the lift range.* The range leaves out AutoLR (summed lifts only) and GEARS (anonymised metrics; its Metric 1 lifts would lower the floor to +0.011%); its lower end is DMRL's main-table value (other tables go down to +0.021%). Without RecEvolve's user satisfaction and EvoPilot's GSRR, the two metrics least clearly broad, or without PILOT's non-randomised before/after comparison and A/B Agent's selecting-round figure, the range is the same over 15 loops.
- *F9, borderline online-selecting loops.* Without GEARS, which makes one selection pass per experiment (§5.1), the F9 counts are 0 of 9 (1 partial) and 1 of 9 (3 partial); without GEARS, CORAL and DMRL, which have no explicit keep/reject rule, 0 of 7 (1 partial) and 1 of 7 (2 partial).

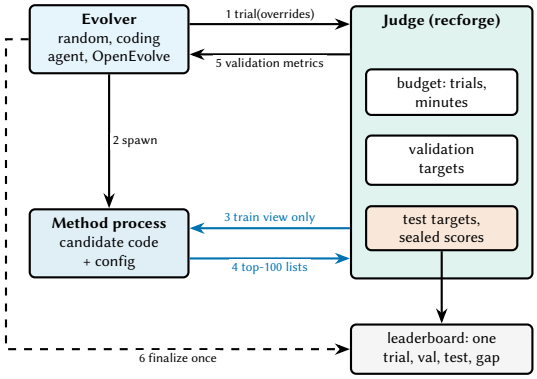


Figure 4: An evolve session in recforge. The evolver requests trials (1); each trial runs the candidate in its own process (2), which receives a training view without targets (3) and returns ranked lists (4). Only validation metrics flow back (5); test scores are computed and sealed. One `finalize` publishes a single trial with its validation score, test score, gap and budget (6). The seal holds at the API level; file-system isolation is planned.

F recforge: A Reference Implementation

recforge [143] is an open benchmark that we are building to enforce the parts of the checklist that software can enforce. It is under development, it has produced no evolver results, and the audit does not depend on it. Its design follows the audit (Figure 4).

- *One protocol per version.* The current protocol is leave-one-out per user (last item test, second-to-last validation) with full-catalogue ranking, the user's history removed, and HR and NDCG at 10 and 20 and MRR@100 (items ranked below 100 score zero). Leave-one-out ignores the global timeline and can leak future information [57, 67]; a global temporal split is the next protocol version. Results from different versions are never put in one table, and every result is keyed by the source hash of the method, its configuration and the code commit.
- *A sealed test at the API level.* Method code runs in its own process and receives only a training view; asking for a validation or test target raises an error, and the method returns ranked lists that the parent process scores. The seal holds only against code that uses the API: sealed scores and session state are ordinary files, so an agent with file-system access could read test scores or alter the budget. A container mode that would close this gap (R8) is planned, not implemented.
- *Budgeted evolve sessions.* A session has a trial and time budget. Each trial returns validation metrics only; test metrics are computed and sealed. A single `finalize` reveals the test metrics of exactly one trial, publishes it with the session's budget, and closes the session.
- *Evolvers on equal terms.* An equal-budget random search (the R5 null), a wrapper for an external coding agent, and an OpenEvolve adapter drive sessions through the same interface, and each run reports the validation-to-test gap of its finalized trial. They are tested on synthetic data only; the OpenEvolve adapter has not been run end to end.
- *Baselines and data.* Popularity, ItemKNN [151], BPR [146], SASRec [72], HSTU [205] and TIGER [141], with MovieLens-1M [53], Amazon 2014 Beauty, Sports and Toys [119], MIND news [180], and the OpenOneRec release [225].

Not yet implemented: the container mode, recording of LLM tokens and cost, automatic aggregation of repeated sessions, and the temporal split.